Sistemas Paralelos
7600060

Carlos Antônio Ruggiero

Agosto de 2026
Última atualização - agosto de 2026

Índice

1 Pipeline **7**

1.1 Definição . . . 7
1.2 Pipeline Linear . . . 7
1.3 Desempenho de Pipelines . . . 10
1.4 Classificação de Pipelines . . . 10
1.5 Organização de Memória Entrelaçada . . . 11

2 Processamento Vetorial **15**

2.1 Conceitos Gerais . . . 15

3 Paralelismo **19**

3.1 Introdução . . . 19
3.2 Máquinas Paralelas . . . 19
3.2.1 Granularidade . . . 20
3.2.2 Distribuição de Trabalho . . . 21
3.2.3 Localidade . . . 21
3.2.4 Comunicação . . . 21

4 Paralelismo II **23**

4.1 Objetivos da Computação Paralela . . . 23
4.2 Problemas da Computação Paralela . . . 23
4.2.1 Número de Processadores . . . 23
4.2.2 Comunicação . . . 24
4.2.3 Controle de Tarefas . . . 24
4.2.4 Natureza das Aplicações . . . 24
4.2.5 Confiabilidade . . . 24
4.3 Medição de Paralelismo . . . 25
4.4 Exemplos de Aplicações para Arquiteturas Paralelas . . . 25
4.4.1 Banco de Dados . . . 25
4.4.2 Paralelismo em Inteligência Artificial . . . 27
4.4.3 Sistemas de Produção e Sistemas Especialistas . . . 27
4.4.4 Paralelismo em Sistemas Especialistas . . . 28
4.4.5 Espaços de Estados . . . 28

5 Paralelismo e Modelos Computacionais **31**
5.1 Modelos Computacionais e Algorítmos 31
5.2 Visão Operacional 31
5.3 Classificações 32
5.3.1 Classificação de Flynn 32
5.3.2 Classificação de Treleaven 33
5.4 Visão Analítica 34

6 Análise de Algorítmos **35**
6.1 Introdução 35
6.2 Notação Assimptótica 36
6.3 Análise de algorítmos em Modelos Paralelos 36
6.4 Exemplo de Análise 38
6.4.1 Análise Dependente do Tamanho 38
6.4.2 Análise independente do Tamanho 39

7 Modelo de Fluxo de Dados ("Dataflow") **41**
7.1 Resumo Histórico 41
7.2 Filosofia Fluxo de Dados 41
7.3 O Modelo de Fluxo de Dados 42
7.4 Diferentes Modelos de Fluxo de Dados 44

8 A Máquina de Fluxo de Dados de Manchester **47**
8.1 Introdução 47
8.2 Arquitetura 47
8.2.1 Formato das Fichas 48
8.2.2 Formato das Instruções 48
8.3 Exercício 49

9 Software para Fluxo de Dados **51**
9.1 Introdução 51
9.2 Crítica de Linguagens von Neumann 51
9.3 Linguagens de atribuição única 52
9.3.1 SISAL 52

10 Problemas do Modelo de Fluxo de Dados **55**
10.1 Introdução 55
10.2 Código Sequencial 55
10.3 Estrutura de Dados 55
10.4 Uso excessivo de memória 56
10.5 Depuração 56

Lista de Figuras

1.1 Um pipeline 8
1.2 Diagrama espaço contra tempo num pipeline 8
1.3 Somador de 4 estágios 9
1.4 Acesso S 12
1.5 Acesso C 13

2.1 Paralelismo em vários estágios de compilação 17

4.1 Base de Dados Relacional 26
4.2 Quebra-cabeças 29

5.1 Classificação de Flynn 33
5.2 Classificação de Treleaven 33

7.1 Execução no modelo de fluxo de controle 43
7.2 Grafo de $a = (b + 1) * (b - c)$ 44
7.3 Execução de um grafo de fluxo de dados 45
7.4 Arquiteturas de fluxo de dados 46

8.1 Arquitetura da MDFM 47
8.2 Múltiplos EP 49

Capítulo 1

Pipeline

1.1 Definição

Em seu livro "An Inquiry into the Nature and Causes of the Wealth of Nations", o primeiro dos "liberais" capitalistas mostra como a divisão de trabalho entre vários operários leva a uma produção muito maior do que se todos executassem todos os passos da tarefa. Isto é devido a especialização de cada trabalhador. Este conceito levou à criação da "linha de montagem" tão popular na indústria no século XX.

Em computação, o conceito de "linha de montagem" levou ao de "pipeline". No começo dos anos 60, as arquiteturas Stretch e do sistema Atlas implementaram os primeiros pipelines, nos EUA e Inglaterra respectivamente.

O princípio de funcionamento do pipeline é o seguinte: divide-se um trabalho em várias fases. Cada fase age sobre o produto da fase anterior e é executada por um hardware especializado. O ganho é grande pois muitas atividades ocorrem simultaneamente quando o trabalho é executado sobre um grande número de dados. Em inglês, usa-se o termo "overlap" para descrever esta situação. Este tipo de concorrência é as vezes chamada de "paralelismo temporal".

1.2 Pipeline Linear

Funciona exatamente como a linha de montagem industrial. Cada unidade do hardware age sobre o dado transformado por todos os estágios anteriores.

Um esquema de pipeline pode ser visto na figura 1.1.

Idealmente, cada estágio S levará tempos iguais. L são os latches. A interdependência entre as várias unidades de trabalho $T_1 \dots T_k$ é dada pelo *grafo de precedência*. No pipeline linear, nenhum estágio T_j pode começar até que todos os T_i com $i < j$ tenham sido executados.

O tempo de relógio (ciclo) é dado por:

$$\tau = max\{\tau_i\} + \tau_l = \tau_m + \tau_l$$

onde τ_l é o tempo do latch.

Para melhor entender o ganho em tempo na execução de um "pipeline", suponha que uma tarefa T subdividida em T_i com $1 \leq i \leq k$ tenha que ser efetuada sobre n dados.

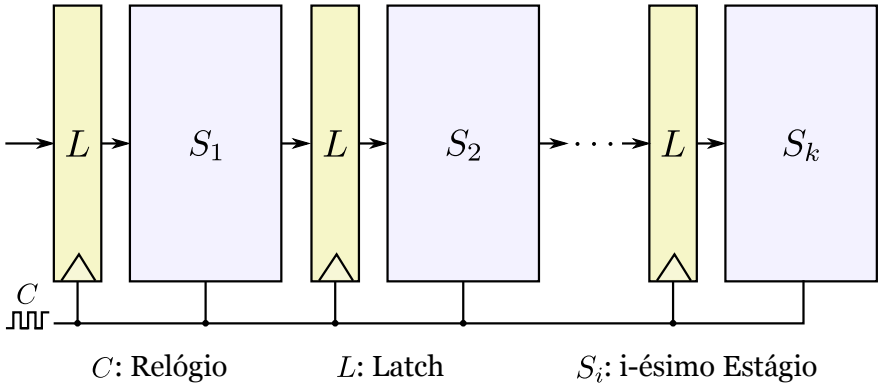


Figura 1.1: Um pipeline

A execução pipeline pode ser desenhada como na figura 1.2 (diagrama de espaço contra o tempo).

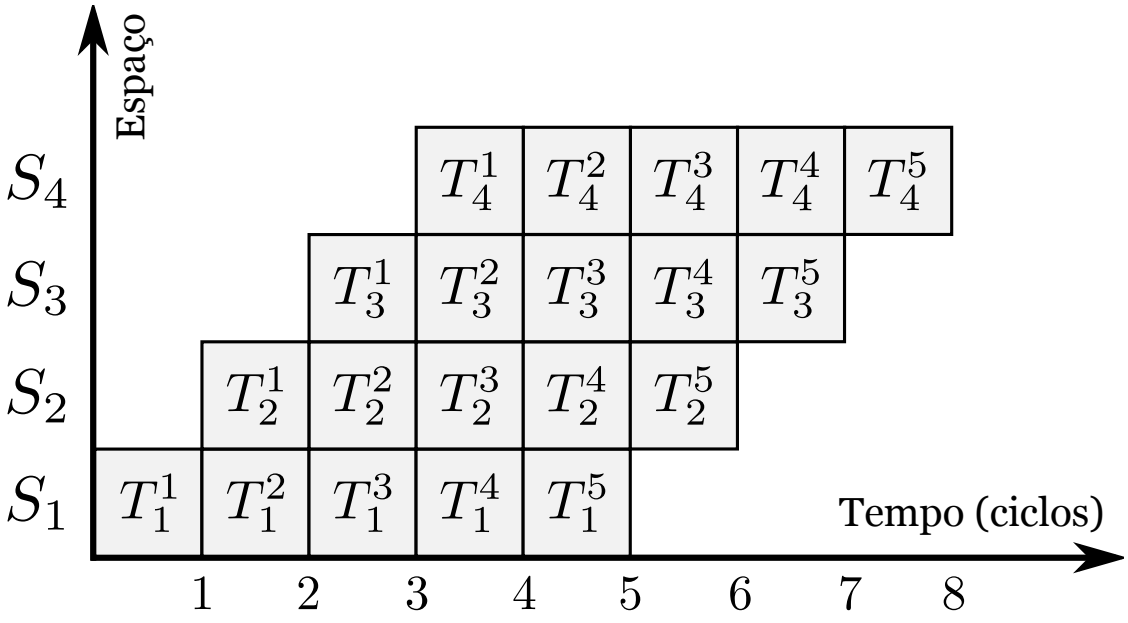


Figura 1.2: Diagrama espaço contra tempo num pipeline

Note que entre 3 e 4 ciclos, 4 atividades ocorrem ao mesmo tempo (agindo sobre dados diferentes). Um pipeline de k estágios recebendo n dados levará T ciclos para ser executado onde T é:

$$T = k + n - 1$$

Se o processamento fosse feito sem pipeline teríamos $T = k.n$. O ganho em velocidade, dado pelo "speedup" é então:

$$Speedup = \frac{kn}{k+n-1}$$

Na fórmula anterior, se $k \gg n$ então o speedup é aproximadamente n enquanto que se $n \gg k$, o speedup fica próximo de k.

Quando o número de dados a ser tratado, n é grande o suficiente, o ganho em termos de tempo é o ideal pois conseguimos k vezes mais velocidade com k vezes mais hardware (aproximadamente).

Como exemplo de pipeline, considere o somador de 4 estágios mostrado na figura 1.3.

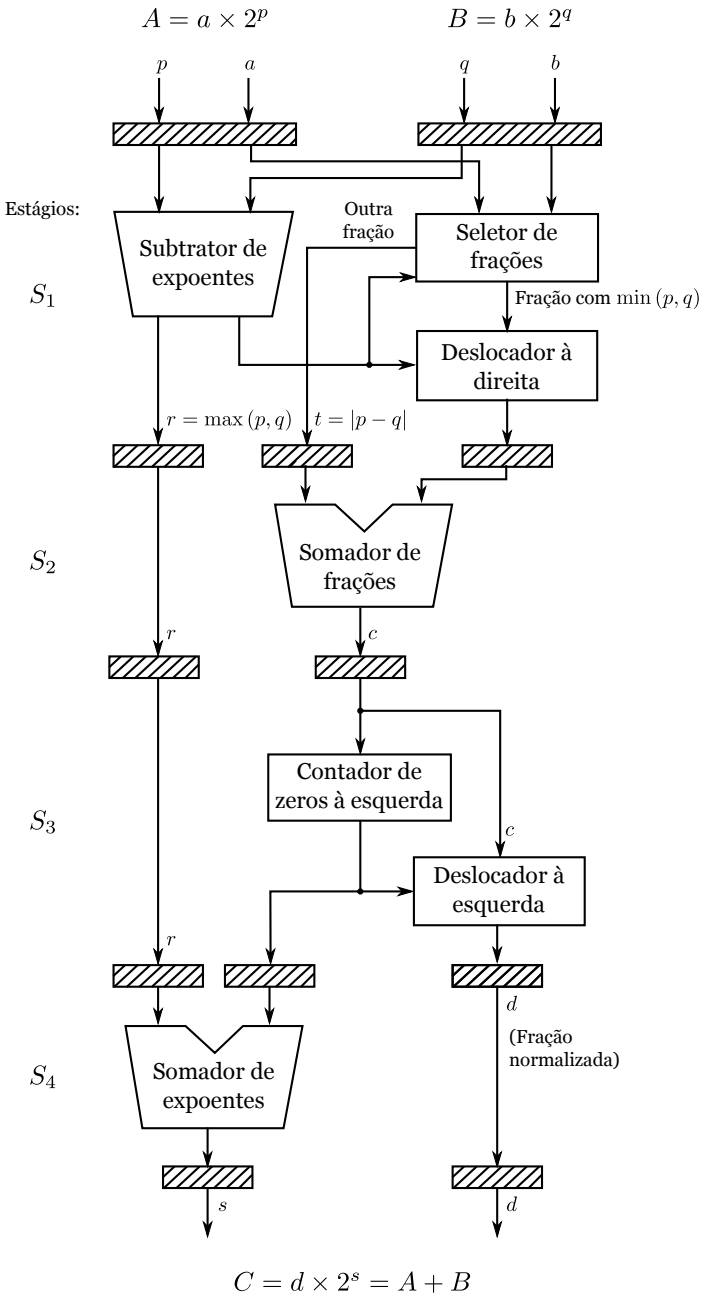


Figura 1.3: Somador de 4 estágios

Suponha-se que se queira somar dois números A e B dados por $A = a \times 2^p$ e $B = b \times 2^q$, onde a e b são duas frações e p e q, os expoentes. A soma em ponto flutuante pode ser feita:

- Se $p \geq q$ faça $t = p - q$. Senão, $t = q - p$
- Se $p \geq q$ rode b de t bits para a direita. Senão rode a, t bits para a direita.
- Some a com b produzindo c.
- Conte o número de zeros u a esquerda em c e rode c a esquerda u bits.
- Corrija o expoente final fazendo: Se $p \geq q$ então $s = p - u$. Senão $s = q - u$

O resultado é então $c = d \times 2^s$.

Suponha que os 4 estágios do pipeline possam ser implementados em: 60ns, 50ns, 90ns e 80ns. Assim, o ciclo deve ser 90ns mais o tempo do latch, digamos 10ns. Tem-se então $\tau = 90 + 10 = 100ns$. Pode-se usar uma frequência de $f = 1/100ns = 10Mhz$.

Se utilizássemos um circuito sem pipeline teríamos $60 + 50 + 90 + 80 = 280 + 2latches = 300ns$. O ganho do pipeline é então $300/100 = 3$

1.3 Desempenho de Pipelines

A eficiência de um pipeline (η) pode ser dada por

$$\eta = \frac{Speedupatingido}{SpeedupIdeal} = \frac{n}{k+n-1}$$

Além da eficiência, outra medida bastante usada é o "Throughput" (w) que mede o número de dados de saída do pipeline na unidade de tempo. Assim, n dados saem num tempo dado por $(k + n - 1)\tau$. Então

$$w = \frac{n}{(k+(n-1))\tau} = \frac{\eta}{\tau}$$

No caso ideal tem-se $w = 1/\tau = f$

1.4 Classificação de Pipelines

De acordo com o nível de processamento, pipelines podem ser divididos em:

- Pipeline aritmético: A ULA de uma UCP pode ser feita em forma de pipeline (como no exemplo dado anteriormente). Exemplos de ULA em pipeline são: a ULA do Star-100 (4 estágios), a ULA de oito estágios usada no TI-ASC e a ULA de até 14 estágios do CRAY-1.
- Pipeline de instruções: A execução de uma instrução é feita concorrentemente com a busca, decodificação e busca de operandos de instruções subsequentes. Praticamente todas as UCP modernas (incluindo microprocessadores) tem alguma forma de pipeline de instruções. Em alguns casos, como no 8086, o pipeline é simplesmente um prefetch sendo que as próximas instruções são buscadas e armazenadas num buffer interno a UCP. Em arquiteturas mais avançadas, como no processamento RISC, o pipeline é "completo".

- Pipeline de processadores: Várias UCP agem uma após as outras sobre uma sequência de dados de entrada. As UCP se comunicam umas com as outras através de uma memória normal compartilhada (todas têm acesso).

De acordo com Ramamoorthy e Li (1977), os pipelines podem ser classificados de acordo com sua configuração e controle em:

- Unifunção versus Multifunção: um pipeline dedicado a uma função específica como o exemplo de pipeline dado (somador) é chamado de unifunção. O pipeline com mais de uma função é chamado de multifunção.

- Estático versus dinâmico: um pipeline é dito estático quando sua configuração não muda com o tempo. Se a configuração for variável, chama-se o pipeline de dinâmico.

- Pipelines escalares versus vetoriais: Pipelines escalares agem sobre dados escalares como operandos de instruções internas a um ciclo DO do Fortran. Em pipelines vetoriais, instruções vetoriais agem sobre os vários elementos de uma estrutura de dados. Processadores contendo pipelines vetoriais são chamados de processadores vetoriais.

1.5 Organização de Memória Entrelaçada

Largura de Banda de uma determinada organização de memória é definida como o número médio de palavras por unidade de tempo que a organização pode fornecer. A largura de banda de uma organização de memória é o parâmetro principal na avaliação dessa organização. O problema de conseguir-se um sistema de memória eficiente é frequentemente encontrado em arquiteturas paralelas. O problema surge da seguinte forma: projetam-se unidades funcionais com enorme capacidade de processamento; se estas unidades forem muito rápidas (o que é o objetivo!), o tempo gasto em alimentá-las com dados passa a representar uma alta porcentagem do tempo total de processamento; como os dados são armazenados em algum tipo de memória, é fundamental garantir-se que o sistema de memória não acabe por limitar a capacidade de processamento.

O principal método de organização de memória com grande largura de banda é conhecido como *entrelaçamento*.

Dois tipos de acesso à memória são utilizados para aumentar a largura de banda de um sistema de memória: o acesso S e o acesso C.

No acesso S (de simultâneo), todos os 2^m módulos de memória são acessados simultaneamente com a aplicação de $n - m$ linhas de endereçamento a todos. Os m bits menos significativos do endereço são usados para selecionar a saída do módulo correspondente através de um multiplexador, como pode ser visto na figura 1.4.

Já no acesso C (de concorrente), que pode ser visto na figura 1.5, os $n - m$ bits mais significativos de endereço são aplicados sequencialmente a latches de entrada dos vários módulos de memória. Posteriormente, os vários dados são lidos também sequencialmente, dos m módulos. Este método só é vantajoso quando o tempo de ciclo das memórias é bem maior que o tempo de acesso.

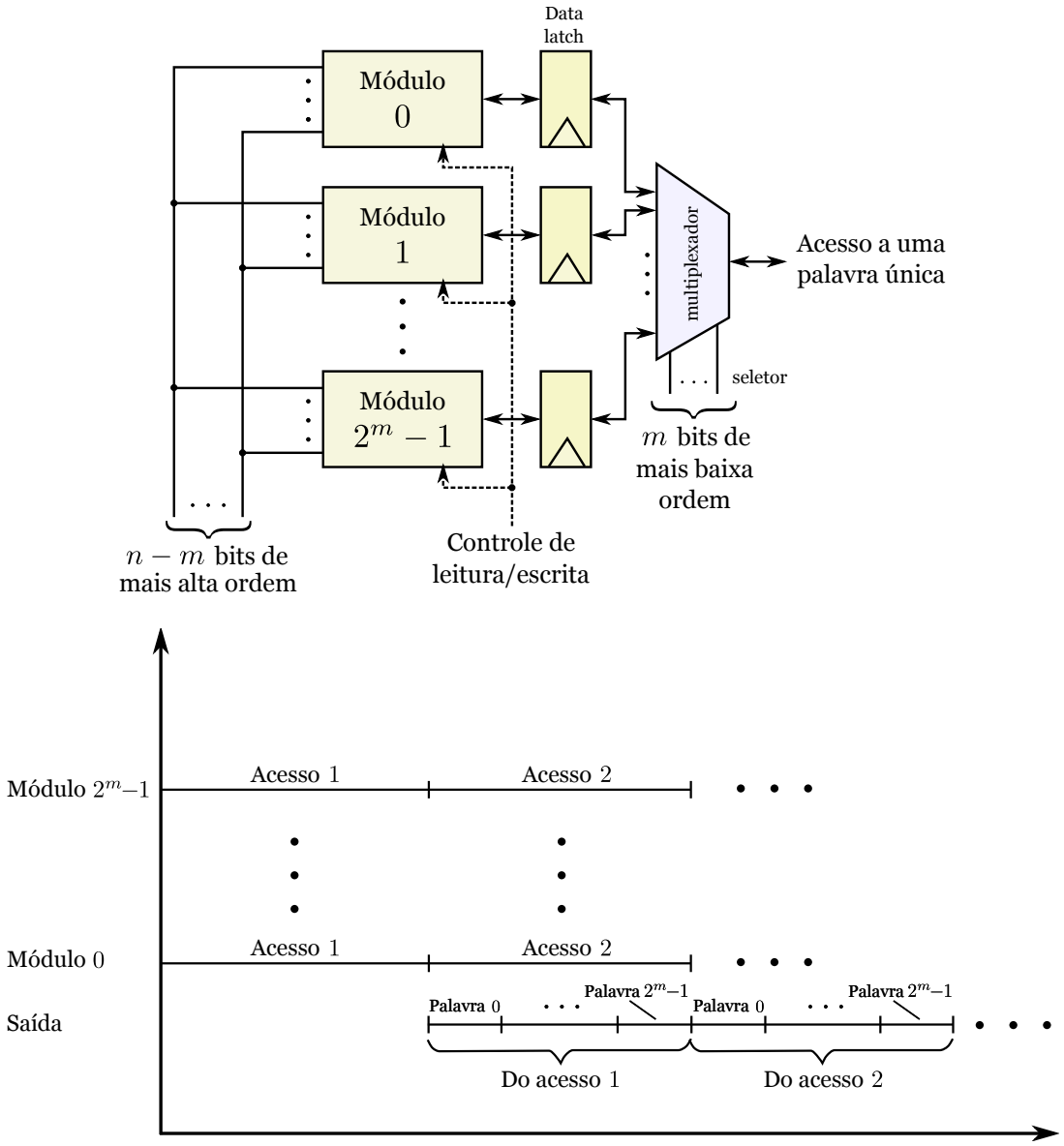


Figura 1.4: Acesso S

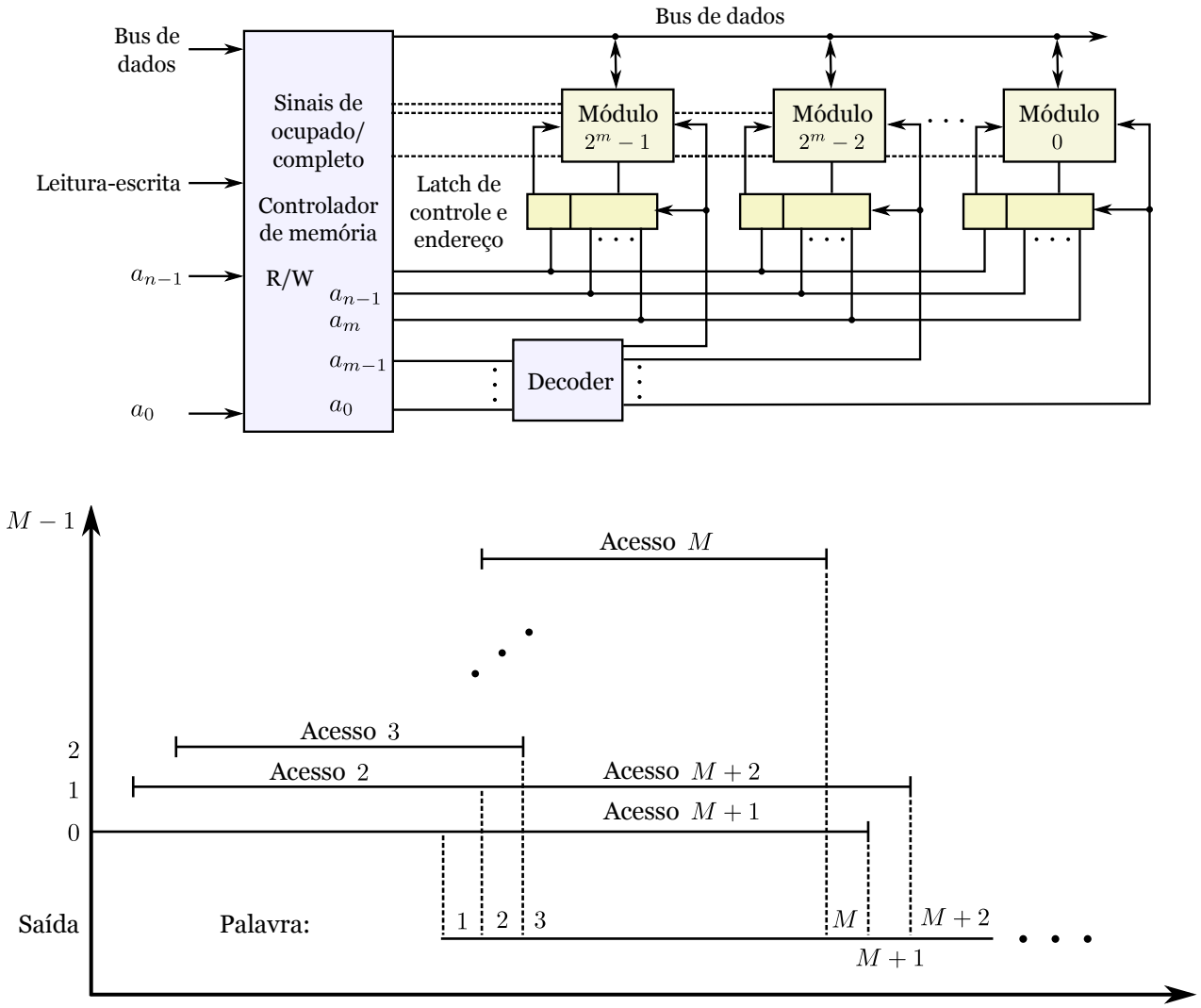


Figura 1.5: Acesso C

Capítulo 2

Processamento Vetorial

2.1 Conceitos Gerais

Uma quantidade escalar é aquela que é totalmente especificada por um único número. Em computação, pode ser um inteiro, um valor lógico, um valor em ponto flutuante, um caracter ASC, etc. Quantidades vetoriais, por outro lado, precisam de n elementos (n escalares) para sua completa especificação. Os n elementos são organizados de maneira ordenada, um após o outro, o que lhes vale também o nome de arranjo ("array").

As funções vetoriais mais importantes são as unárias e as binárias. Funções unárias são aquelas tais que

$$f_1 : V \to V$$

ou

$$f_2 : V \to S$$

No primeiro caso, toma-se um único vetor e se produz um outro vetor (por exemplo, o vetor de saída é a raiz quadrada do vetor de entrada). No segundo, o vetor de origem dá lugar a um valor escalar. Exemplos de f_2 são: a função que soma os elementos de um vetor e a função que retorna o elemento de maior valor do vetor.

Funções vetoriais binárias tomam dois argumentos (um dos quais é um vetor) e produz um vetor na saída. Os dois tipos a seguir são as funções básicas.

$$f_3 : V \times V \to V$$

ou

$$f_4 : V \times S \to V$$

Os exemplos de f_3 são vários: soma de dois vetores (soma elemento a elemento), multiplicação de dois vetores (multiplicação elemento a elemento), "e" lógico entre dois vetores, etc. Como exemplo de f_4, pode ser citada a multiplicação de cada elemento de um vetor por um dado escalar.

Além das operações acima discutidas, algumas instruções existem em computadores vetoriais para facilitar o trabalho com vetores. Cumpre citar a instrução "compare" que gera um vetor booleano a partir da comparação dos elementos A[I] e B[I] de dois vetores A e B. Outra instrução comum é a instrução "compress" que comprime um vetor de acordo com uma máscara vetorial. Máscara vetorial é um vetor que contém elementos lógicos. Assim, se o valor de M[I] (M é o vetor máscara) for verdadeiro, o elemento A[I] é incluido no vetor de saída. O comprimento do vetor de saída é então igual ao número de valores "verdadeiro" em M. Finalmente, a instrução "merge" combina dois vetores num único vetor de acordo com um vetor máscara. Como exemplo de um merge, considere dois vetores $A = (1, 2, 4, 8)$ e $B = (3, 5, 6, 7)$ e seja um vetor máscara definido por $M = (1, 1, 0, 1, 0, 0, 0, 1)$. O vetor final será (1,2,3,4,5,6,7,8).

Num computador vetorial moderno, uma instrução vetorial envolve os seguintes campos:

- O código do operando.
- O endereço base e o "offset".
- O incremento do endereço.
- O comprimento do vetor a ser utilizado.

Processadores vetoriais podem ser divididos em dois tipos: memória a memória e registrador a registrador. No tipo de memória a memória, todos os operandos vetoriais são tomados diretamente da memória. Exemplos destas arquiteturas são o TI-ASC, o CDC STAR-100 e o CYBER-205. No modelo de registradores, os operandos são trazidos da memória para os registradores e a UCP só age sobre estes registradores. Neste caso estão os computadores da CRAY e o Fujitsu VP-200.

Como exemplo de operação vetorial, considere o código FORTRAN abaixo:

```
      DO 100 I=1,N
      A[I] = B[I] + C[I]
  100 B[I]=2*A[I+1]
```

Em um processador vetorial, o código gerado será:

```
TEMP(1:N) = A(2:N+1)
A(1:N) = B(1:N) + C(1:N)
B(1:N) = 2*TEMP(1:N)
```

Note a introdução do vetor TEMP para possibilitar a vetorização.

Comparando-se os dois códigos, nota-se que no escalar existe considerável perda de tempo no controle do ciclo ("loop"). Por outro lado, as máquinas vetoriais perdem algum tempo até "acertarem" os vetores nos seus lugares e começarem realmente as operações ("setup time").

Um problema comum encontrado em processadores vetoriais esta relacionado ao comprimento dos vetores envolvidos. Em geral, vetores devem ter comprimento suficiente para que haja algum ganho em relação ao tratamento escalar. Porém, se forem muito longos, o tempo perdido na partição do vetor pode ser considerável.

A maior crítica em relação ao processamento vetorial é a dificuldade de se projetar um compilador eficiente que consiga "vetorizar" suficientemente o programa do usuário (que normalmente é escrito em FORTRAN).

De fato, a situação encontrada atualmente em máquinas vetoriais é, no mínimo, curiosa. Suponha-se que se comece com um algorítmo com alto grau de paralelismo. Então, o programa é codificado em FORTRAN, uma linguagem tipicamente sequencial. Então, o compilador deve gerar um código que reflita o paralelismo existente no algorítmo (e que foi perdido na codificação FORTRAN). É claro que idealmente, a linguagem não deveria "esconder" o paralelismo existente no algorítmo. A figura 2.1 mostra gráficos de paralelismo contra estágios de compilação no caso ideal e no caso do compilador vetorial.

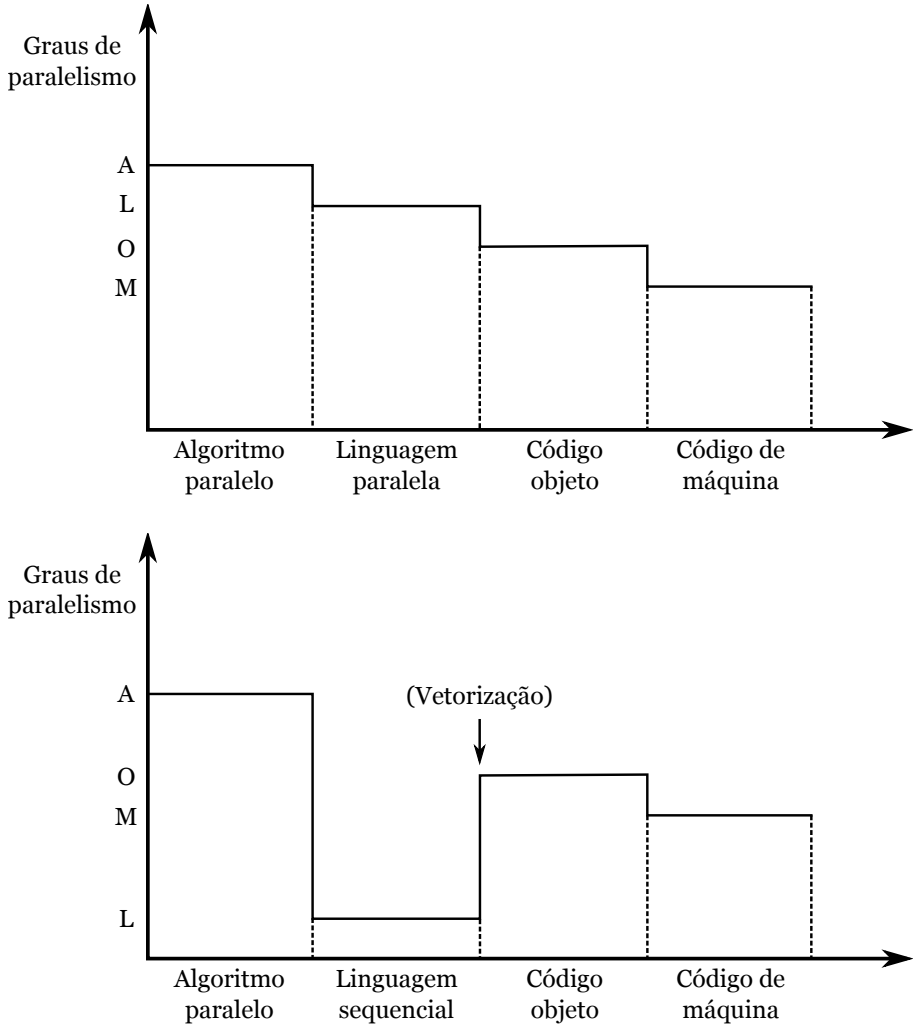


Figura 2.1: Paralelismo em vários estágios de compilação

Capítulo 3

Paralelismo

3.1 Introdução

O motivo fundamental que estimula o estudo do paralelismo é a busca por máquinas de alto desempenho. Limitações físicas (velocidade da luz e princípio da incerteza) já não permitem que circuitos com velocidades de comutação muito mais altas que as atuais sejam construídos (nem muito mais compactos).

Várias arquiteturas têm sido propostas visando explorar o paralelismo disponível em muitos algorítmos. A maneira convencional (ou "von Neumann") de se explorar paralelismo é através de processadores vetoriais com o auxílio de compiladores "vetorizadores". Várias arquiteturas não von Neumann têm sido propostas como alternativas para os processadores vetoriais tais como arquiteturas de fluxo de dados ("dataflow") e de redução.

3.2 Máquinas Paralelas

A exploração de paralelismo pode ser efetuada em vários níveis:

- Programa ou job.
- Subrotina ou procedimento.
- Instruções.
- Intra-instruções.

A exploração de paralelismo a nível de programa não é muito difícil já que normalmente, programas não se comunicam muito entre si. A exploração de paralelismo a nível de intra-instrução por outro lado, é bem executada em arquiteturas von Neumann convencionais. Um exemplo é a nova filosofia de projeto RISC que paraleliza as várias fases de execução de uma instrução (paralelismo temporal).

Uma classificação útil de máquinas paralelas refere-se aos tipos de programa que se supõe que ela deva executar. Assim, máquinas podem ser:

- De Propósito Geral: A máquina pode ser usada numa grande variedade de programas e aplicações, eficientemente.

- Máquinas dedicadas: Seu uso é restrito a uma aplicação ou a uma classe de aplicações.

Máquinas dedicadas normalmente apresentam uma relação custo/desempenho melhor do que máquinas de propósito geral. Porém, construir várias máquinas dedicadas quando uma grande variedade de programas deve ser executada pode ser muito caro. Neste caso, uma máquina de propósito geral é melhor.

As propriedades ideais de uma máquina de propósito geral são:

1. Linguagem de Propósito Geral: Deve haver pelo menos uma linguagem capaz de expressar as várias formas de computação. O código produzido pela linguagem deve ser eficiente e capaz de explorar todo o potencial do hardware.

2. Hardware simples: O hardware não deve ser nem complexo nem caro em relação a seu desempenho. Um bom padrão para referência é uma máquina sequencial.

3. Expansibilidade Linear: Supondo que o hardware é composto de N elementos processadores, a adição de um elemento extra deveria:

   - Implicar em pouca modificação de hardware.
   - Implicar em pouca ou nenhuma modificação no software.
   - Melhorar a velocidade do hardware por $(N + 1)/N$ quando existir paralelismo suficiente.

É fácil ver que estas propriedades não são fáceis de serem conseguidas. De fato, até agora, nenhuma máquina construída ou proposta atingiu esses objetivos. Apesar disso, estas "propriedades ideais" são úteis na avaliação de qualquer máquina paralela de propósito geral.

Alguns conceitos importantes relacionados às propriedades acima são explicados a seguir. Estes conceitos são de fundamental importância no entendimento da situação atual de pesquisa em máquinas paralelas.

3.2.1 Granularidade

Granularidade é um conceito relacionado ao nível de paralelismo que pode ser explorado pelo hardware. Se uma máquina aloca grandes processos a processadores, ela é dita ser de *granularidade grossa*. Exemplos de máquinas de granularidade grossa são o Cm* e o Denelcor HEP, máquinas multiprocessadoras ditas convencionais.

Uma máquina é dita de *granularidade fina* se ela aloca pequenas unidades de trabalho aos vários processadores. Em particular, máquinas que alocam instruções a processadores, como as de fluxo de dados, são consideradas de granularidade bastante fina. Máquinas de granularidade fina são linearmente expansíveis porque elas "expõe" e exploram níveis muito altos de paralelismo. Isto significa que o número N de elementos de processamento pode ser alto sem degradação de desempenho. A distribuição de trabalho (veja a seguir) nestas máquinas é normalmente boa, mas existe uma excessiva quantidade de comunicação entre os vários elementos processadores. A relação custo desempenho pode não ser tão boa.

Máquinas de granularidade grossa, por outro lado, tendem a apresentar boa relação custo desempenho devido a eficiente execução de "pedaços" sequenciais de código. Entretanto, a

expansibilidade linear não é facilmente atingível devido a sua inabilidade de explorar todo o paralelismo disponível.

Existem máquinas de granularidade mais grossa que a de fluxo de dados mas mais fina que a de multiprocessadores convencionais. A idéia é atingir uma melhor relação entre vantagens e desvantagens entre os dois enfoques. Um exemplo é a máquina de redução Flagship.

3.2.2 Distribuição de Trabalho

Distribuição de trabalho é um dos itens mais importantes no estudo de máquinas paralelas e implica na distribuição das tarefas igualmente entre os vários processadores.

Uma distribuição de trabalho eficiente é mais fácil de atingir em máquinas de granularidade fina porque aqui, os métodos estocásticos funcionam melhor. Esta é uma consequência direta do fato de existir mais trabalho a ser distribuído. Por outro lado, tarefas em máquinas de granularidade grossa apresentam complexidade procedural e, consequentemente seu tamanho é imprevisível. Torna-se difícil então distribuí-las igualmente pelos processadores.

3.2.3 Localidade

Dois conceitos de *localidade* podem ser distinguidos: a localidade *espacial* e a localidade *temporal*. Diz-se que uma determinada arquitetura explora localidade espacial (ou temporal) se instruções e processos relacionados entre si são executados em processadores próximos uns dos outros (ou em tempos próximos).

A exploração de localidade pode diminuir a quantidade de comunicação requerida entre processadores. Entretanto, se impomos que uma linguagem sempre produza localidade a ser explorada, a linguagem pode perder sua generalidade.

3.2.4 Comunicação

Quando paralelismo a nível de instrução ou de processo é explorado, existe uma considerável quantidade de comunicação entre processadores. Máquinas de granularidade fina apresentam uma necessidade maior de comunicação entre processadores do que máquinas de granularidade grossa. Na verdade, em máquinas como as de fluxo de dados, a comunicação entre processadores é proporcional ao número de instruções executadas.

A quantidade de comunicação existente numa unidade de tempo é chamada de *largura de banda (bandwidth)*. Máquinas que exigem altos valores para largura de banda tendem a ser caras e complexas.

Capítulo 4

Paralelismo II

4.1 Objetivos da Computação Paralela

Os objetivos de um processador paralelo podem ser resumidos nos 3 ítens dados a seguir:

- Tentar executar com maior rapidez, programas que mesmo nos supercomputadores mais rápidos são executados muito lentamente. Programas típicos desta área são simulação de circuitos (lógicos em microeletrônica, previsões metereológicas, projetos auxiliados por computador, problemas de dinâmica molecular, cálculos em QCD ("quantum chromodynamics"), sistemas de bancos de dados etc...

- Executar programas que rodem suficientemente rápido em supercomputadores em ambientes que não dispõe destes dispositivos.

- Aumentar a produtividade de programadores quando escrevendo código para aplicações intrinsicamente paralelas.

As maiores limitações encontradas por arquiteturas sequenciais vêm da tecnologia (projeto, microeletrônica). Acredita-se que arquiteturas paralelas poderão superar arquiteturas sequenciais inclusive em termos de relação custo/desempenho (segundo ítem acima) num futuro próximo (note que arquiteturas sequenciais apresentam ótimas relações de custo/desempenho).

4.2 Problemas da Computação Paralela

Para atingir seus objetivos, a computação paralela deverá resolver uma série de problemas dos quais os mais importantes são discutidos a seguir.

4.2.1 Número de Processadores

Não é claro até o presente instante, quantos processadores devem ser utilizados num computador paralelo. É natural supor-se que um número sempre maior de processadores deverá resultar numa máquina cada vez mais poderosa. Isto, porém, não é necessariamente verdade visto que o "overhead" de comunicação também aumenta com o número de processadores podendo levar a uma máquina com relação custo/desempenho bastante pobre.

4.2.2 Comunicação

O problema de comunicação entre processadores aparece em praticamente qualquer arquitetura paralela proposta. Os problemas fundamentais podem ser resumidos em:

- Que tipo de informação deve ser trocada entre os vários processadores?
- Qual a porcentagem de comunicação em relação ao tempo de processamento?
- A comunicação entre processadores deve ser síncrona ou assíncrona?

4.2.3 Controle de Tarefas

Os problemas aqui são:

- Qual é o tamanho de cada tarefa (granularidade)?
- Como se deve controlar a execução de todas as tarefas nos vários processadores?
- Qual a relação da execução das tarefas e o sistema operacional da máquina?

4.2.4 Natureza das Aplicações

Não é claro que tipos de aplicações devem ser executadas em ambientes paralelos. Assim, deve-se resolver os seguintes problemas:

- Que algorítmos devem ser utilizados em arquiteturas paralelas?
- Qual a importância relativa de máquinas de propósito geral e máquinas dedicadas.
- Como o programador verá a máquina? Quais as facilidades que ele terá para implementar uma determinada aplicação (compiladores, editores de texto, depuradores, etc...)

4.2.5 Confiabilidade

Computadores paralelos possuem várias unidades funcionais (de processamento). É possível que algumas destas unidades falhem e seria interessante que a máquina não parasse completamente como é o caso de máquinas sequenciais com somente um processador. De fato, a máquina deveria ser capaz de continuar executando uma aplicação (obviamente com certo prejuízo no desempenho) mesmo quando fosse detectado que várias unidades funcionais não estão funcionando corretamente. Apesar de que várias propostas de arquiteturas paralelas tolerantes a falha tenham sido feitas, nenhuma até o presente momento foi universalmente aceita.

4.3 Medição de Paralelismo

Não existe até o presente momento, nenhuma maneira universalmente aceita para se quantificar o paralelismo existente nos vários estágios do software. É normalmente reconhecido que o paralelismo existente num determinado algorítmo decresce nos vários estágios do software (programa, compilação, sistema operacional, código de máquina), de forma que, muitas vezes um algorítmo paralelo pode acabar se transformando num código quase que puramente sequencial.

O conceito normalmente utilizado para medição de paralelismo é bastante informal: "A quantidade de paralelismo é dada pelo número médio de operações que podem ser executadas simultaneamente". O conceito de "operação" na definição acima fica bastante vago pois ele pode apresentar granularidade bastante diferente nos vários estágios do "software".

4.4 Exemplos de Aplicações para Arquiteturas Paralelas

4.4.1 Banco de Dados

Um banco de dados é uma coleção de dados que representam fatos, e que são organizados para facilitar o acesso a informação. É normal que banco de dados sejam demasiadamente grandes para serem mantidos na memória principal. Assim, é fundamental um sistema rápido de memória de massa para sua eficiente implementação.

Uma transação num banco de dados envolve dois passos:

- Decifrar o que o usuário quer
- Obtenção do dados.

Exploração de paralelismo na decifração parece muito promissora. Já na obtenção do dado (entrada e saída) não parece haver muito espaço para tal exploração.

No processamento de uma transação (primeiro ítem), há basicamente duas fontes de paralelismo: entre transações de usuários diferentes e dentro de uma transação complexa única. A fonte de paralelismo que deve ser explorada depende muito da aplicação sendo executada. Assim, num banco de dados bancário, por exemplo, as transações dos vários usuários são simples e independentes entre si. Duas transações (como dois usuários tentando verificar seus saldos) podem ser facilmente executadas em paralelo. O paralelismo explorável é da ordem dos milhares. O paralelismo numa única transação complexa (por exemplo, "todos os fornecedores em São Paulo que mantém tinta azul metálica para o Monza") esta na ordem de centenas ou poucos milhares.

Como exemplo de exploração de paralelismo em banco de dados, considere o modelo relacional de bases. Neste modelo, os dados são organizados em tabela e a recuperação de informação envolve a construção de uma tabela contendo os dados pertinentes.

Considere a figura 4.1 dada a seguir.

Existem três operações em bases de dados relacionais:

- Seleção: constrói uma tabela que contém somente algumas linhas da tabela original.
- União: junta duas tabelas em uma tabela mais larga de acordo com alguma condição.

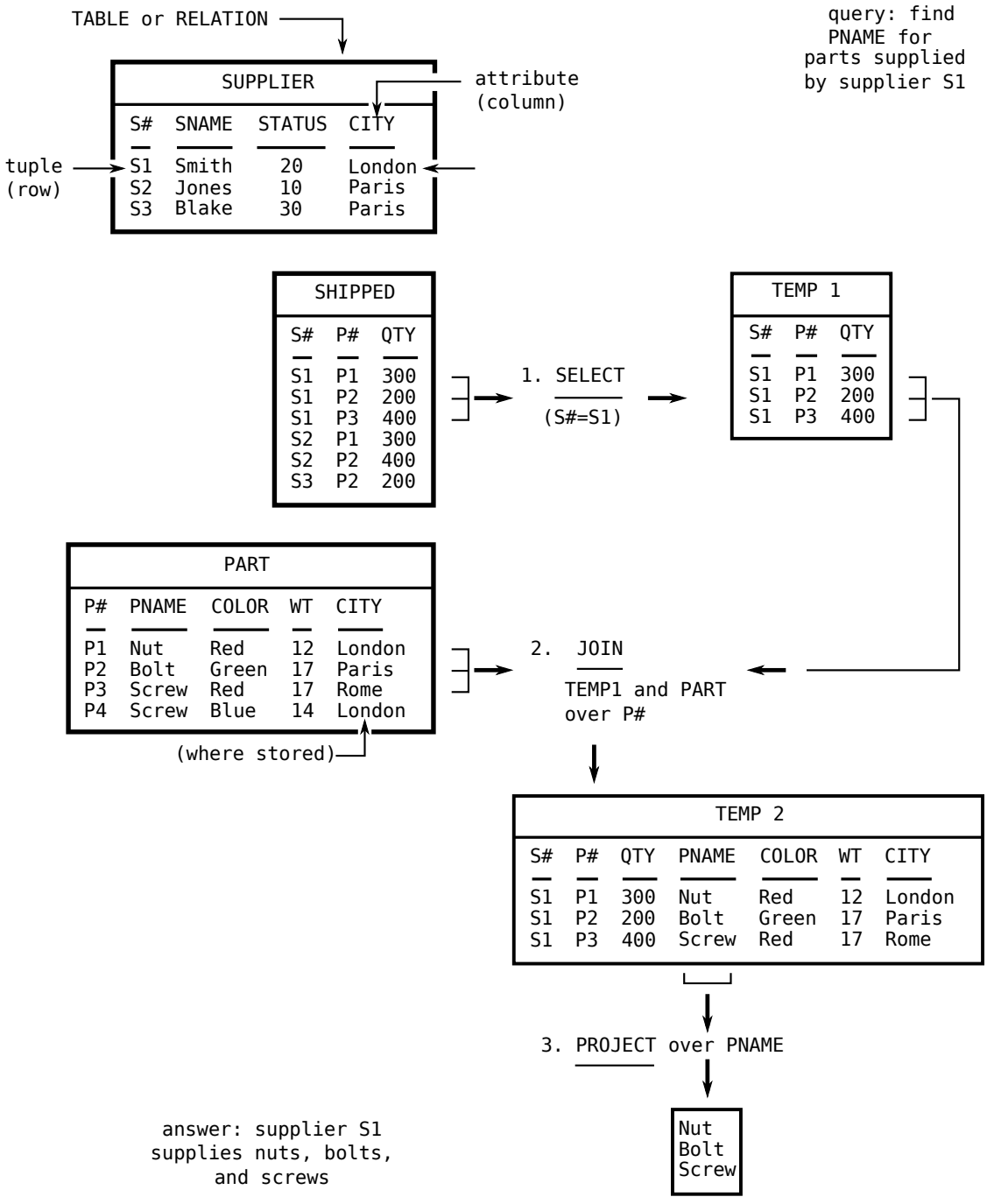


Figura 4.1: Base de Dados Relacional

- Projeção: extrai algumas colunas das tabelas e remove repetições.

Na operação de seleção, o paralelismo que pode ser explorado é óbvio: cada processador pode verificar se a condição é satisfeita para um subconjunto das colunas.

Na operação de Projeção, o mesmo pode ser feito mas a eliminação de linhas comuns deve ser global.

A operação de União pode também ser paralelizada como a Seleção mas aqui, certos cuidados precisam ser tomados. Na União, cada linha de uma tabela deve ser testada contra todas as linhas da outra tabela. Isto pode levar muito tempo. É claro que uma das tabelas pode ser dividida em várias e cada parte entregue a um processador diferente que testará sua parte da tabela contra todas as linhas da outra tabela. O esquema pode ser otimizado de tal forma a separar "ambas" as tabelas tentando reduzir o trabalho de cada processador. É claro que as partes disjuntas das tabelas não devem ter nenhum "match" na propriedade especificada. Esta divisão de tabelas, porém, deve ser feita com cuidado para garantir uma conveniente distribuição de trabalho entre os vários processadores.

4.4.2 Paralelismo em Inteligência Artificial

Existem muitas atividades que o computador desempenha melhor que seres humanos. Um exemplo é o cálculo numérico em que o computador consegue ser milhões de vezes mais rápido. Existem, no entanto, certas atividades nas quais os seres humanos são muito melhores que qualquer computador construído até o presente. O objetivo do estudo de IA é exatamente melhorar o desempenho das máquinas nestas áreas.

Em IA, os dois componentes mais importantes são conhecimento e raciocínio. Aplicações atuais de IA são:

- Visão Artificial (Reconhecimento de Padrões).
- Sistemas Especialistas.
- Processamento de Linguagens Naturais.
- Resolução de problemas e Planejamento.

4.4.3 Sistemas de Produção e Sistemas Especialistas

Muito do conhecimento humano em determinados assuntos pode ser reduzido a uma série de regras (que podem chegar aos milhares) do tipo "se *condição* então ...". Se aplicarmos este conjunto de regras a dados reais para inferir algo sobre o mundo real, temos um *sistema de produção*. Se as regras fazem parte do conhecimento de um ser humano que é considerado um especialista na área, temos um sistema especialista.

Um sistema de produção envolve:

- Uma memória de trabalho ou uma base de dados global que contém dados reais.
- Uma memória de produção que contém o conjunto de regras ("se condição então ..."). Estas regras agem sobre a base de dados global e, quando a condição é satisfeita, a cláusula é efetuada (a parte do então...) podendo inclusive modificar a memória de trabalho.

- Estratégias de Controle: que controlam quando e quais regras de controle devem ser aplicadas.

Um sistema de produção fica continuamente executando os seguintes passos:

- Match: elimina todas as regras que não são satisfeitas pelos dados na memória de trabalho.
- Seleção: seleciona, dentre as regras restantes após o match, as regras a serem aplicadas segundo um sistema de controle pré-definido.
- Ação: as regras escolhidas agem sobre a memória de trabalho.

4.4.4 Paralelismo em Sistemas Especialistas

O trabalho da estratégia de controle envolve muito a ação de "procurar" ("search"). De fato, praticamente todas as aplicações em inteligência artificial contém consideráveis quantidades de operações buscando chegar ou encontrar uma resposta para um determinado problema. Para se chegar a esta resposta, muitos "caminhos" devem ser percorridos e extensivamente vasculhados. Existe portanto, um grande potencial para a exploração de paralelismo pois vários processadores podem trabalhar concorrentemente nos vários caminhos que possam lever à solução do problema. O que ocorre é análogo ao problema de se procurar uma criança perdida numa floresta; quanto mais pessoas participarem da busca, mais rapidamente a criança será achada (e mais provável que ela seja achada viva!!).

4.4.5 Espaços de Estados

Todos os estados possíveis de um problema formam o espaço de estado do problema. Achar uma solução significa a partir de um dado estado, chegar a um outro estado com características especificadas. Considere o quebra-cabeças mostrado na figura 4.2.

O espaço de estados total do problema são todas as posições que podem ser atingidas movendo qualquer peça (número) para o espaço em branco. O objetivo aqui é chegar a um estado específico (os números em sequência circular com o espaço em branco no centro. No caso de sistemas especialistas para diagnóstico médico, poder-se-ia dizer que os estados são asserções sobre o paciente; o objetivo é produzir uma asserção do tipo *o paciente têm ....* Para sair de uma asserção à outra, usam-se regras de produção (no caso do jogo, as regras de produção são as regras de movimentação de números).

É preciso muito cuidado na busca de uma solução visto que em muitas aplicações existe o perigo da "explosão combinatória" que se traduz pelo fato de serem tantas as possibilidades que é simplesmente impossível se varrer todas elas. Um exemplo típico é encontrado no jogo de xadrez.

Voltando ao quebra-cabeças, temos que o espaço dos estados contém 9! (362880) configurações possíveis (estados). As regras são que os números a direita, a esquerda, acima ou abaixo do espaço em branco podem se mover para ocupar o dito espaço. O espaço ocupa então a posição do número deslocado.

Para resolver o problema, pode-se propor que:

1. Nunca se volta a um estado pelo qual já se passou.

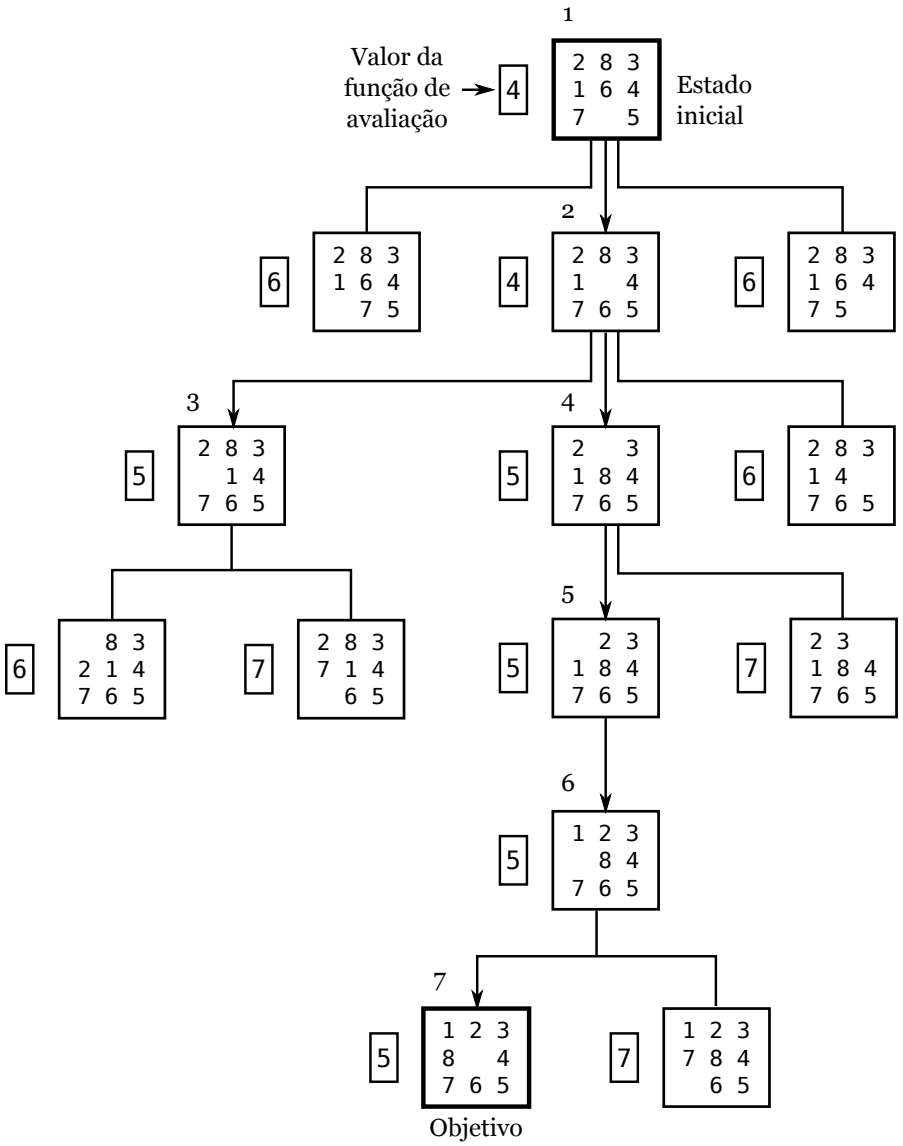


Figura 4.2: Quebra-cabeças

2. Em qualquer estado, o movimento escolhido é aquele que minimiza a função de avaliação. A função de avaliação é $f(n) = W(n) + d(n)$ onde n é o estado, W(n) é o número de números posicionados erroneamente e d(n) é o número de estados desde o estado inicial.

A busca "guiada" como esta sugerida acima é chamada de busca *heurística*.

Capítulo 5

Paralelismo e Modelos Computacionais

5.1 Modelos Computacionais e Algorítmos

A expressão modelo computacional é usada para descrever algo um pouco mais abstrato do que a arquitetura de um computador; na verdade, o "modelo" inclui as premissas fundamentais comuns a um determinado conjunto de arquiteturas. Assim, modelo computacional pode ser definido como o conjunto de regras que são seguidas no projeto de uma determinada classe de arquiteturas. Existem dois aspectos no modelo computacional: o lado operacional que trata das possíveis ações que uma arquitetura aderente ao modelo pode executar, quando são essas ações executadas e como os dados podem ser armazenados e recuperados. O lado operacional responde à pergunta "Quais são as regras do jogo?".

O outro lado do modelo computacional permite a "análise" de algorítmos e permite que se estime quão eficiente um algorítmo poderá executar numa arquitetura aderente ao modelo.

5.2 Visão Operacional

Um modelo computacional deverá incluir:

- Ações básicas que o computador pode executar.
- A maneira que os dados são armazenados e como eles podem ser acessados (*mecanismo de dados*).
- A maneira como as ações a serem executadas são controladas *mecanismo de controle.*
- Os modos de comunicação entre as várias unidades de processamento (quando existirem mais de uma).
- Mecanismos de sincronização entre as unidades de processamento (também, quando houver mais de uma).

O modelo computacional mais bem conhecido é aquele utilizado nos computadores comerciais o qual foi proposto por von Neumann.

No modelo de von Neumann, as características são:

- As instruções são do tipo multiplique este dado por aquele e coloque o resultado em tal lugar.
- Os dados são armazenados juntamente com o programa num espaço linear de armazenamento. Esse espaço contém cédulas com endereço único.
- As instruções são executadas, uma após a outra, e são buscadas na memória sequencialmente.

As principais críticas a esse modelo são:

- A existência de uma ligação (UCP à memória) que é sobrecarregada (dados e instruções passam por ela) e facilmente se torna um gargalo.
- A existência de "efeitos colaterais", pois a memória de dados se mistura com a memória de programas.

5.3 Classificações

5.3.1 Classificação de Flynn

A classificação mais comum de modelos é aquela sugerida por Flynn em 1966. Nessa classificação existem 4 modelos fundamentais:

- SISD (Single Instruction Single Data): é o modelo von Neumann convencional. Nesse modelo, existe sempre uma única instrução agindo sobre um único dado num instante de tempo qualquer.
- SIMD (Single Instruction Multiple Data): aqui uma única instrução age sobre um conjunto de dados simultaneamente. Nessa classe incluimos os processadores de arranjo ("array") e os processadores de vetores ("vector").
- MIMD (Multiple Instruction Multiple Data): neste caso, estão os computadores onde múltiplas instruções agem sobre múltiplos dados. A maioria das pesquisas em arquitetura paralela se concentram hoje nesta categoria.

Na figura 5.1 vê-se a classificação completa proposta por Flynn. Note a inclusão de MISD que implicaria num conjunto de instruções agindo sobre um único dado. Não existe um grande interesse no estudo de MISD, apesar de alguns pesquisadores acreditarem que computadores "pipeline" estejam nesta categoria.

O exemplo clássico de máquinas SIMD é o ILLIAC-IV, que contém um conjunto de unidades funcionais iguais onde cada uma recebe um elemento de uma estrutura. Cada instrução é enviada para todos os procesadores que agem sobre os dados simultaneamente. O uso de múltiplas unidades funcionais, porém, pode se tornar muito cara e uma solução mais barata (?) pode ser encontrada em projetos do tipo CRAY-1 onde um pipeline substitui o arranjo de processadores. Neste último caso o speedup máximo que se consegue é o número de estágios do "pipeline". Classificamos ambos os projetos como SIMD, usando os termos SIMD paralelo no primeiro caso e SIMD pipeline no segundo.

		Número de dados	
		Único	Múltiplos
Número de instruções	Única	SISD (von Neumann)	SIMD (vetor, array)
	Múltiplas	MISD (pipeline?)	MIMD (múltiplos micros)

Figura 5.1: Classificação de Flynn

5.3.2 Classificação de Treleaven

Treleaven considera a divisão MIMD dada por Flynn e a subdivide de acordo com o mecanismo de dados e com o mecanismo de controle utilizados.

Dois tipos de mecanismos de dados são considerados: o de memória compartilhada e o de por passagem de mensagens. O de memória compartilhada é o mais parecido com o modelo de von Neumann que o por passagem de mensagem.

Na área de controle, a classificação engloba desde as arquiteturas com controle forte (como os multiprocessadores convencionais) até um controle frágil como o encontrado em máquinas de fluxo de dados. O controle como aquele encontrado em máquinas convencionais é chamado de "control-driven", ou seja, dirigido pelo controle; máquinas de fluxo de dados são chamadas "data-driven", ou seja, dirigidas pelos dados. Entre o "data-driven" o o "control-driven", Treleaven inclue o "demand-driven" (encontrado em máquinas de redução) e "pattern-driven" (lógica).

A figura 5.2 sumariza a classificação de Treleaven.

	Mecanismo de dados	
Mecanismo de controle	Memória compartilhada	Memória privada (passagem de mensagens)
Dirigido por controle	von Neumann	Processos se comunicando
Dirigido por padrões	Lógica	Atores
Dirigido por demanda	Redução de grafos	Redução de cadeias
Dirigido por dados	Fluxo de dados com estrutura-I	Fluxo de dados

Controle menos explícito ↓

Figura 5.2: Classificação de Treleaven

5.4 Visão Analítica

O modelo computacional nos ajuda a avaliar qual o custo da excução de um dado algorítmo numa determinada máquina.

Seja primeiramente o caso do modelo sequencial de von Neumann. Algorítmos são avaliados em termos do aumento verificado no tempo de execução quando se aumenta o tamanho do problema. Considere, por exemplo, o problema de se ordenar n elementos; quer-se determinar o tempo de execução T (ou da memória utilizada S) quando n é aumentado (infelizmente não é sempre que um algorítmo pode ser completamente caracterizado por um único parâmetro).

Na análise de algorítmos, é difícil prever exatamente o comportamento do tempo de execução quando se aumenta o tamanho da aplicação indefinidamente (comportamento assintótico). Assim, normalmente se desprezam as diferenças em termos de constantes. Desta forma, as funções $f(x) = 3x^3$ e $g(x) = 5x^3 + 9$ são considerados equivalentes pois para $x \geq 3$ temos que $f(x) < g(x)$ e $g(x) < 2f(x)$. Esta equivalência é simbolizada pela letra grega θ.

Capítulo 6

Análise de Algorítmos

6.1 Introdução

Costuma-se dizer que o aumento do desempenho de um computador na execução de um determinado programa pode vir de quatro razões distintas:

- Melhoria do algorítmo utilizado para resolver o problema.
- Melhoria do sistema de compilação conseguindo um código de máquina mais eficiente.
- Melhoria da interface entre software e hardware.
- Melhoria do hardware através de tecnologias mais rápidas, melhores arquiteturas, múltiplos processadores, etc.

Dentre todas essas possibilidades, a que pode levar a maiores ganhos é a melhoria do algorítmo. De fato, ganhos de várias ordens de grandeza podem ser conseguidos. Exemplos clássicos da evolução conseguida pela melhoria de algorítmos são os casos de algorítmos para ordenação rápida ("shellsort" e "quicksort") e o famoso algorítmo para FFT que tormou viável uma série de aplicações em processamento de sinais.

A análise de algorítmos para máquinas sequenciais, até o tempo presente, tem-se limitado a contagem de operações executadas em função do tamanho do conjunto de dados. O módelo computacional assumido é aquele convencional ou de von Neumann. Admite-se que o tempo de execução do programa (e a ocupação de memória) é proporcional ao número de instruções executadas.

Dois grandes problemas surgem imediatamente: em primeiro lugar a decisão que um algorítmo é melhor que o outro não pode ser extendida para modelos não convencionais paralelos; em segundo lugar, o número de instruções executadas não é mais uma boa medida do tempo de execução devido à mudança radical na relação do tempo gasto em operações aritméticas e o tempo de transporte de dados (comunicação).

Na análise de algorítmos, duas fases são levadas a cabo:

- Devem ser determinadas quais as instruções que devem ser consideradas na análise; instruções aritméticas, comparações, etc. Note que certas instruções podem ter tempo de execução que dependa do tamanho dos dados sobre os quais age (exemplo: comparação de duas cadeias).

- Determinar vários conjuntos de dados que façam com que o algorítmo exiba seu melhor desempenho, seu pior desempenho e um desempenho médio. Completo entendimento do algorítmo é necessário nesta fase.

Duas fases da análise podem ser distinguidas: a análise *a priori* e a análise *a posteriori*. Na análise a priori, determina-se uma função que limita o tempo de execução do programa. Na análise a posteriori, colhem-se dados estatísticos executando o programa para vários conjuntos de dados.

Para se determinar a priori o tempo de execução de um algorítmo, é preciso determinar qual a frequência das instruções e qual o tempo de cada instrução. Como o tempo de cada instrução é dependente da máquina, fica-se normalmente só com a frequência das instruções. A análise de algorítmos preocupa-se somente com ordens de magnitude e constantes são consideradas irrelevantes.

6.2 Notação Assimptótica

A principal notação matemática utilizada na análise de algorítmos é a notação O.

Diz-se que $f(n) = O(g(n))$ se e somente se existirem duas constantes positivas c e n_0 tais que $|f(n)| \leq c|g(n)|$ para todo $n \geq n_0$. Se f(n) é o tempo de execução de um algorítmo onde n pode ser o teamanho dos dados, então g(n) representa um limite superior no tempo de execução do algorítmo, ou seja, sabemos que para n suficientemente grande, f(n) serÁ menor que uma constante vezes g(n). Como exemplo, podemos mostrar que o tempo de execução do algorítmo de multiplicação de matrizes comum é $O(n^3)$. Ordens de grandeza comuns em algorítmos são $O(1), O(logn), O(n), O(nlogn), O(n^2), O(n^3) e O(2^n)$.

Note que, se um algorítmo for $O(2^n)$, então não importa qual seja a constante, ele só poderá ser executado se n for suficientemente pequeno.

Diz-se que $f(n) = \Omega(g(n))$ se e somente se existirem constantes positivas c e n_0 tais que para todo $n > n_0$, $|f(n)| \geq |g(n)|$.

Se $f(n) = O(g(n))$ e $f(n) = \Omega(g(n))$, então $f(n) = \theta(g(n))$. Note que tanto nos algorítmos de ordenação quanto nos de cálculo da transformada de Fourier, os algorítmos rápidos são $O(nlogn)$ enquanto os lentos são $O(n^2)$.

6.3 Análise de algorítmos em Modelos Paralelos

Sempre na análise de algorítmos, é fundamental definir-se qual o modelo sobre o qual a análise se apóia. Quando se está supondo um modelo sequencial, a escolha óbvia é o modelo de von Neumann. No caso de modelos paralelos porém, tem-se pelo menos dois modelos que devem ser levados em consideração; o modelo com memória compartilhada e o modelo por passagem de mensagens. No modelo com memória compartilhada, tem-se uma memória global que pode ser acessada por todos os processadores. Vários modelos existem, dependendo da estratégia adotada quando mais de um processador deseja ler ou escrever de uma mesma posição de memória: o modelo CREW ("concorrent read exclusive write") permite leitura simultanea por todos os processadores, mas só permite uma escrita por vez. O modelo mais restrito é o EREW enquanto o mais aberto é o CRCW que ficou conhecido como *paracomputador*.

Nos modelos de memória compartilhada, é comum supor-se que qualquer acesso à memória leva o mesmo tempo independentemente de P, o número de processadores o que é certamente irreal

Nos modelos de passgem de mensagem, supõe-se que cada processador possui sua própria memória de dados a qual nenhum outro processador tem acesso. Deve ser assumido também, um determinado tipo de *grafo de interconexão* que mostra como os processadores estão ligados entre si.

Para se passar da análise de algorítmos sequenciais para modelos paralelos, é fundamental reconhecer que, agora, o tempo não é só dependente do tamanho de dados mas também do número de processadores (T(P,N)). Note que, assim como foi suposto que o tamanho de um algorítmo sequencial podia ser caracterizado por um número N, supõe-se que o tamanho da máquina paralela pode ser bem caracterizado com somente um número, ou seja, P.

Suponha, como exemplo inicial, que o tempo de um algorítmo é limitado por T(P,N) dado por:

$$T(P,N) = N/P + log(P)$$

Esta função não é incomum na análise de algorítmos reais em máquinas paralelas. A fórmula contém dois termos; um refere-se ao processamento local (N/P) onde os N dados iniciais são distribuídos por P processadores e o outro refere-se ao processamento global que promove a combinação dos resultados dos P processadores.

O speedup de um algorítmo é definido por

$$S(P,N) = \frac{T(1,N)}{T(P,N)}$$

e sua eficiência por

$$E(P,N) = \frac{S(P,N)}{P}$$

Algorítmos ótimos são aqueles que conseguem speedup linear, ou seja, proporcionais a P. Estes algorítmos são chamados de *completamente paralelizáveis*.

Como no caso de modelos sequenciais, estamos interessados em saber o comportamento assintótico de algorítmos paralelos, ou seja, veremos o desempenho de T(P,N) para N cada vez maior. Surge, porém, um problema: o que fazer com P? Deve ele permanecer constante ou deve variar com N? É claro que, se P permanecer constante, o máximo que se pode conseguir em relação ao tempo de processamento sequencial é um fator $1/P(f(n)/P)$. Como os fatores constantes são irrelevantes na análise assintótica, os algorítmos ótimos para processamento sequencial serão os mesmos para modelos paralelos. Assim, é fundamental que P varie com N. O próximo problema é como se deve variar P com N? Acontece que, dependendo da variação de P com N, um algorítmo pode ser elhor ou pior que outro.

Dependendo da variação de P e N tem-se diferentes análises: quando N é alguma função de P (e, em particular, qunado $N = P$) o problema é dito *dependente do tamanho*. Se N variar independentemente de P temos um problema *independente do tamanho*.

Define-se o fator de carga L como sendo a razão entre o tamnaho dos dados N e o tamanho da máquina P. Assim, o fator de carga é dado por $L = NP$. Se L for muito grande, diz-se que se está no limite de carga pesada. Note que em carga pesada, tanto N como P crescem só que N é sempre muito maior que P.

Existem vários problemas a se considerar quando da escolha de um algorítmo; não é garantido que um algorítmo com desempenho assintótico melhor realmente executará mais eficientemente em uma dada máquina. A seguir são enumeradas algumas dificuldades que podem surgir.

1. A análise a priori de algorítmos não se preocupa com constantes. Assim, dois algorítmos com mesmo desempenho assintótico podem exibir comportamentos bastante distintos quando colocados a executar numa máquina real. Um exemplo é o método húngaro de ordenação em paralelo. Nesse método, a análise revela que para N processaodres, tem-se complexidade $\theta(logN)$ que é a ideal. Porém, as constantes envolvidas são tão grandes que o algorítmo não pode ser utilizado.

2. Na análise, sempre se consideram os piores casos. Muitas vezes porém, os piores casos dificilmente ocorrem na prática o que significa que o algorítmo é, na verdade, melhor do que o suposto. Um exemplo é a solução "elipsoide" para programação linear proposta por Khachiyan. Trata-se de um método que, no pior caso, é polinomial em N. A solução tradicional de Dantzig (método "simplex") é exponencial em N no pior caso. Assim, pode parecer que o método Khachiyan é muito melhor. Porém, na prática, o crescimento exponencial no método de Dantzig não é observado e o método "simplex" continua o mais poular.

3. Nas análises, a comunicação de dados não é levada em consideração. Este pode ser um problema muito sério principalmente se o modelo adotado é por passagem de mensagens.

6.4 Exemplo de Análise

Seja o problema de se somar N números. A maneira sequencial de fazê-lo é claramente $\theta(N)$. Consideremos agora um modelo de passagem de mensagens com P processadores.

6.4.1 Análise Dependente do Tamanho

Suponhamos que $N = P$, ou seja, cada processador contém um dado a ser somado. Suponha que a topologia de ligação de processadores seja uma árvore binária. Assim, cada nó passa seu dado para o nó pai que soma os resultados dos dois filhos. Os nós "folha" recebem os números a serem somados. Se a árvore tem altura H, tem-se que

$$N = P = 2^{H+1} - 1$$

e vem que $H = \theta(logP)$. É fÁcil ver que $T(P, N) = \theta(H) = \theta(logN)$.
O speedup do algorítmo é dado por

$$S(P, N) = \frac{T(1,N)}{T(P,N)} = \frac{\theta(N)}{\theta(logN)} = \theta\frac{P}{logP}$$

A eficiência será

$$E(P, N) = \frac{S(P,N)}{P} = \theta(\frac{1}{logP})$$

o que é um resultado bastante ruim pois indica que a eficiência cai quando se aumenta P.

6.4.2 Análise independente do Tamanho

Suponha que coloquemos N dados em cada processador de forma que a razão $L = N/P$ possa variar livremente. Agora, o algorítmo começa com os processadores somando seus L valores antes de combiná-los com outros. É fácil ver que, agora, tem-se

$$T(P, N) = \theta(L) + \theta(logP) = \theta(L + logP)$$

Capítulo 7

Modelo de Fluxo de Dados ("Dataflow")

7.1 Resumo Histórico

O termo "Dataflow", ou fluxo de dados, foi primeiro utilizado em duas teses de doutorado escritas em finais dos anos 60 no MIT e na universidade de Stanford nos EUA. Jack Dennis, professor do MIT nos início da década de 70, começou a usar o termo para descrever sua nova arquitetura que buscava superar as limitações impostas pelo modelo von Neumann. A idéia de um modelo que pudesse explorar todo o paralelismo existente numa determinada aplicação, sem que o programador tivesse que explicitá-lo, parece ter surgido para Dennis na sua participação no projeto de sistema operacional Multics (que estava sendo financiado pelo MIT, General Eletric e Bell Labs e que seria a base do sistema Unix, já que Ken Thompsom também trabalhou no projeto Multics).

A década de 70 presenciou um enorme interesse em arquiteturas de fluxo de dados por parte de pesquisadores em ciência da computação no mundo todo. Projetos importantes surgiram, tais como o projeto LAU na França dirigido por Syre, os projetos de fluxo de dados dinâmicos no MIT (dirigido por Arvind) e em Manchester (Watson e Gurd) além do próprio projeto dirigido por Jack Dennis também no MIT que era uma máquina de fluxo de dados estático.

No final dos anos 70, o Japão interessou-se pela nova arquitetura; de fato, os japoneses lançaram o arrojado projeto da quinta geração de computadores e acreditavam que só com o modelo de fluxo de dados, poder-se-ia atingir o poder computacional desejado na área de inteligência artificial.

Atualmente, pesquisa em fluxo de dados tem sido realizada no MIT, em em Manchester, na IBM e no Japão, onde o enfoque é em máquinas para processamento simbólico de alto desempenho.

7.2 Filosofia Fluxo de Dados

Todos os pesquisadores que se envolveram com arquiteturas de fluxo de dados partilham idéias comuns sobre o futuro da computação. Os principais pontos dessa filosofia comum são:

- Só a exploração de paralelismo pode levar a computadores ordens de magnitude mais rápidos que os atuais. A tecnologia está chegando ao seu limite.

- Não se deve deixar para o programador a tarefa de descobrir e expor todo o paralelismo existente nas aplicações. Isso leva a um tempo de desenvolvimento de aplicações muito longo e oneroso.

- No futuro, paralelismo a nível bastante fino deverá ser explorado para que um desempenho realmente alto seja atingido. Também, só a esse nível de exploração de paralelismo pode o programador ser completamente liberado da tarefa de analisar sua aplicação e detetar todo o paralelismo existente.

7.3 O Modelo de Fluxo de Dados

É ilustrativo comparar o modelo de fluxo de dados com o modelo de von Neumann. Para isso, considere a linha de programa $a = (b+1)*(b-c)$ e sua implementação numa arquitetura de von Neumann dada a seguir:

```
LDA  2000H
ADI  1
STA  2003H
LDA  2001H
MOV  B,A
LDA  2000H
SUB  B
MOV  C,A
LDA  2003H
MULT   C
STA  2002H
```

Note a característica estritamente sequencial desse código. O modelo von Neumann, ou de fluxo de controle, caracteriza-se pelo controle absoluto da execução de cada instrução através de um apontador para a próxima instrução a ser executada (o "program counter"). Na figura 7.1 pode-se ver uma representação da execução do código sequencial numa máquina de von Neumann. Também pode-se ver uma execução paralela mantendo-se o modelo de fluxo de controle; utiliza-se de construções do tipo "fork" e "join" para tornar explícito o paralelismo. Note porém, que o que se faz é simplesmente criar vários "fluxos de controle" com vários "program counters". É importante notar que a passagem de dados de uma instrução à outra é efetuada através de posições de memória (veja a utilização da posição 2003H no código von Neumann).

Já no modelo de fluxo de dados, as instruções são executadas tão logo seus operandos estejam disponíveis não existindo qualquer controle centralizado da execução do programa. O programa não é mais representado por uma "sequência" de instruções, uma por linha, como é o caso no modelo de fluxo de controle; agora ele é representado de uma forma gráfica por um "grafo" que simplesmente indica as dependências de dados entre uma instrução e outra. Veja a figura 7.2 que ilustra um possível grafo para a linha de programa dada anteriormente.

No grafo de fluxo de dados, cada nó representa uma instrução e cada aresta representa possíveis caminhos de dados. Assim, os dados, que em fluxo de dados são chamados de "fichas" ("tokens") percorrem as arestas (ou arcos) em direção à instrução seguinte. Na

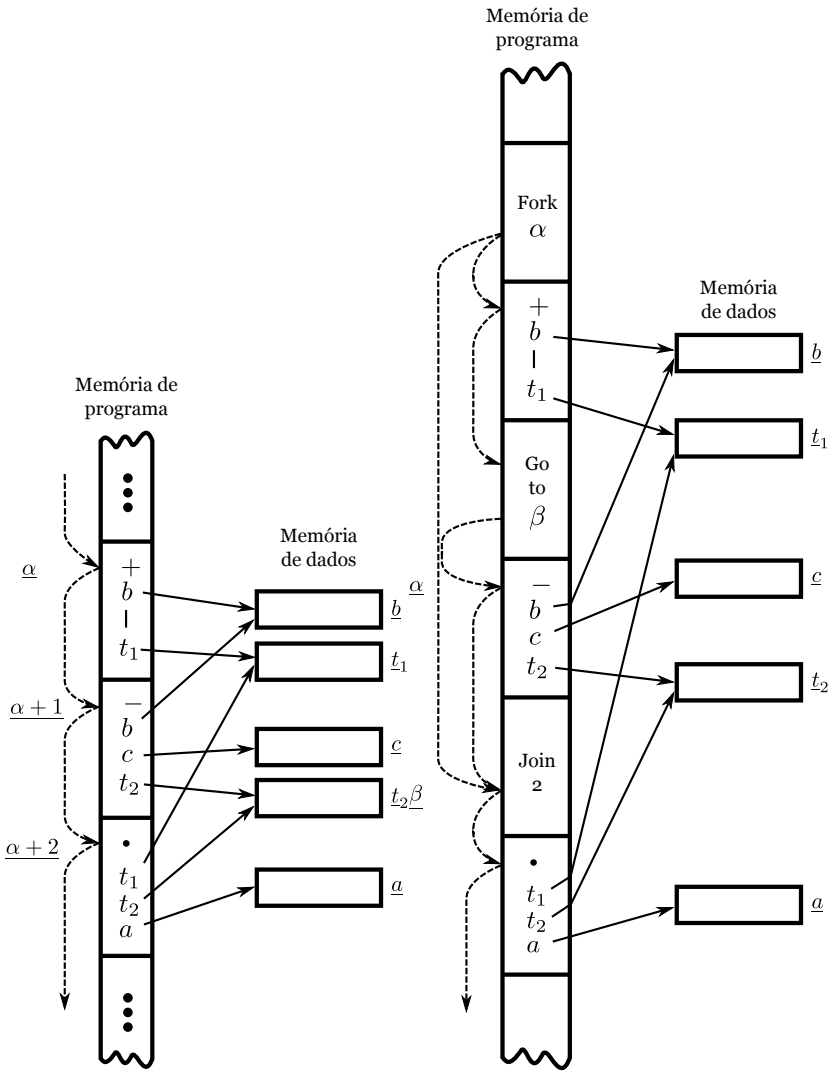


Figura 7.1: Execução no modelo de fluxo de controle

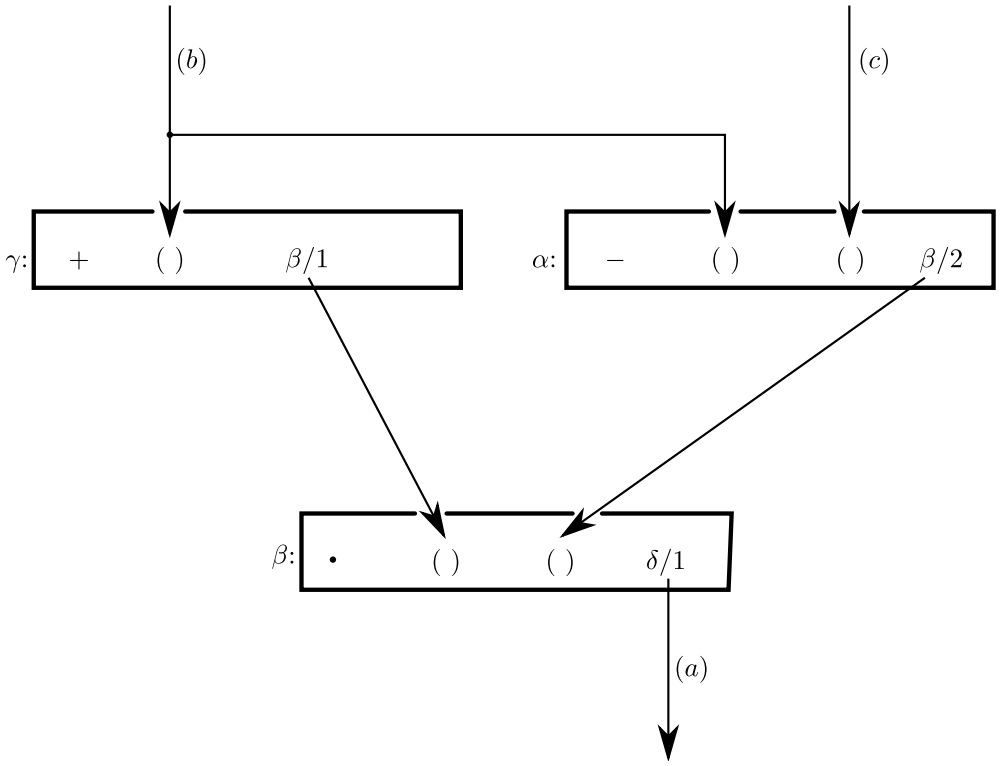


Figura 7.2: Grafo de $a = (b + 1) * (b - c)$

verdade, as fichas são mensagens que são enviadas de um nó ao outro de forma que não existe necessidade de posições de memória globais como no caso do fluxo de controle.

Note na figura 7.2 que mais de uma instrução (a soma e a subtração) podem ser executadas concorrentemente já que não existe dependência de dados entre elas. *Não* é possível saber-se qual das duas instruções executará primeiro; na verdade, em duas execuções diferentes do programa, a ordem de execução das instruções pode ser diferente.

O grafo da figura 7.2 apresenta caminhos possíveis para as fichas e é gerado em tempo de compilação. Em tempo de execução, as fichas percorrem os arcos e os nós são "executados". Uma possível execução do programa anterior pode ser vista na figura 7.3 Quando os dados de um nó (instrução) estão disponíveis, o nó é dito "habilitado" e pode ser executado tão logo haja um processador livre.

7.4 Diferentes Modelos de Fluxo de Dados

Anexo, apresenta-se o artigo de Jack Dennis, que explica os vários modelos existentes em fluxo de dados (cuidado: Jack Dennis trabalha há quase 20 anos com o modelo estático !!!). Como ilustração, vê-se na figura 7.4 duas arquiteturas de fluxo de dados: uma estática e outra dinâmica.

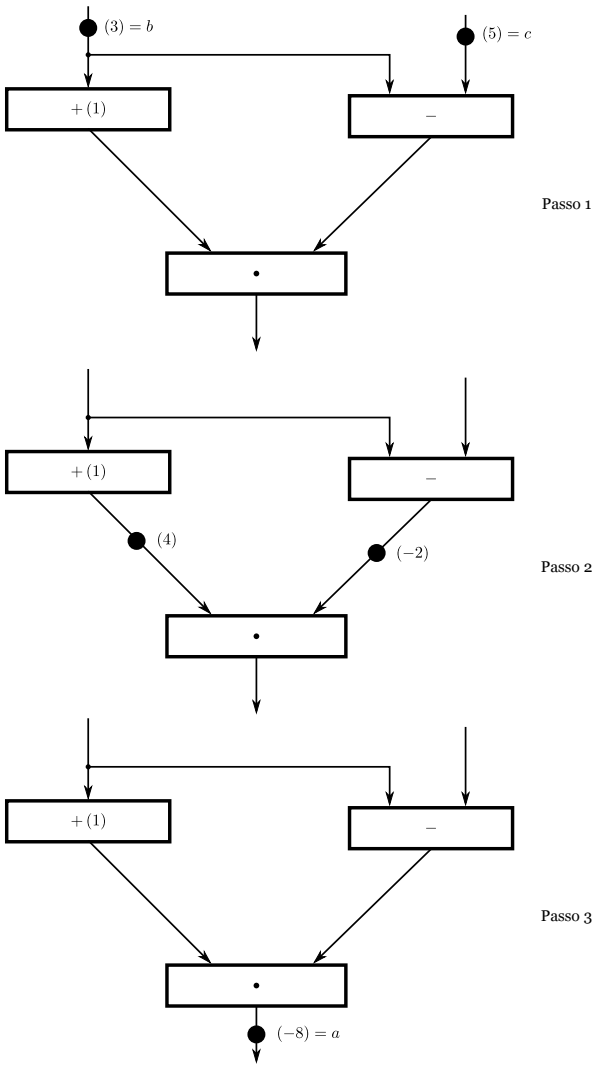


Figura 7.3: Execução de um grafo de fluxo de dados

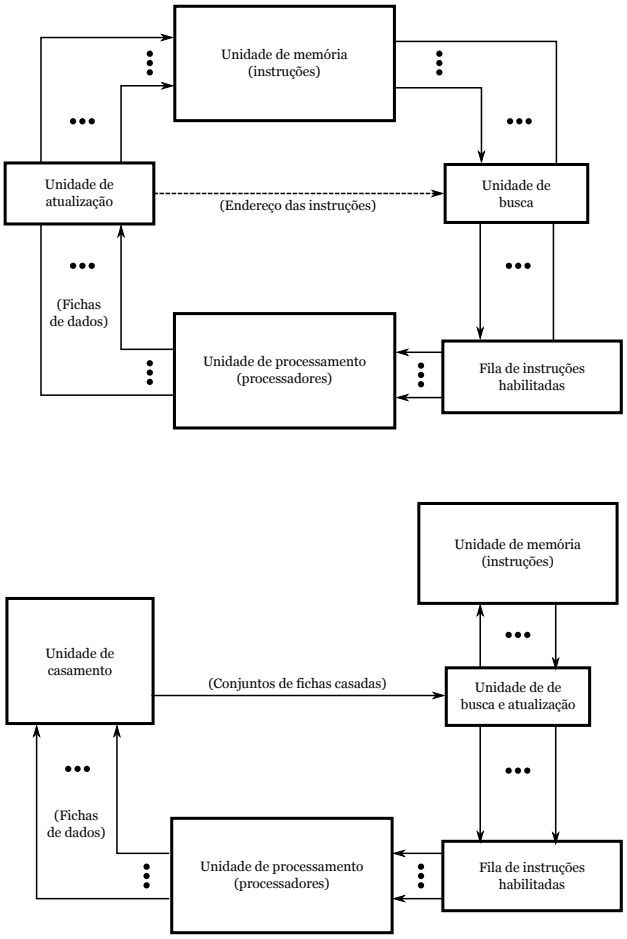


Figura 7.4: Arquiteturas de fluxo de dados

Capítulo 8

A Máquina de Fluxo de Dados de Manchester

8.1 Introdução

Uma das arquiteturas de fluxo de dados mais estudadas na literatura é a máquina de Manchester (MDFM). Trata-se de uma arquitetura dinâmica por rotulação de fichas; consequentemente ela busca explorar todo o paralelismo do programa a nível de instrução. Veremos a máquina de Manchester com algum detalhamento para entendermos melhor os princípios básicos de fluxo de dados.

8.2 Arquitetura

A arquitetura da MDFM pode ser vista na figura 8.1.

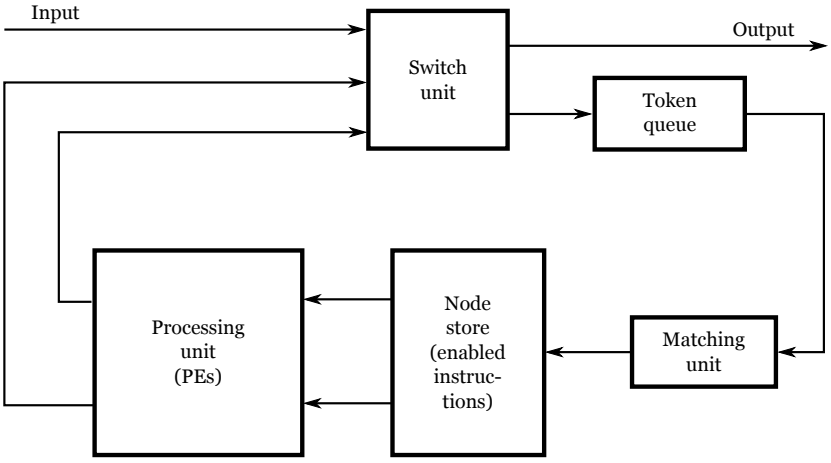


Figura 8.1: Arquitetura da MDFM

Em linhas gerais, o funcionamento da máquina é o seguinte: uma ficha que se encontra na chave é passada para a fila de fichas ou Unidade de Armazenamento (UA). Esta unidade existe apenas e tão somente para regular o fluxo do anel; de fato, como a arquitetura é

circular, deve-se tomar cuidado para se evitar um estado de *deadlock* onde cada unidade está esperando para a próxima liberar sua ficha. A primeira ficha da UA (que é uma ficha) é enviada para a unidade de emparelhamento (UE) que verificará se a ficha se dirige a uma instrução binária ou unária (no código de máquina da MDFM, só existem instruções de uma ou duas entradas). Se ela for para uma instrução unária, sua função de emparelhamento será "BYPASS" e a unidade de emparelhamento enviará a ficha para a unidade de programa (UP). Se ela se dirigir a uma instrução binária, ela conterá uma função de emparelhamento chamada "extract-wait" ou "extrair-esperar" que significa que se a outra ficha que se dirige à mesma instrução já houver chegado à UE, ela será extraída e as duas fichas serão mandadas à UP. Caso contrário, a ficha que chegou é armazenada na UE e espera pela sua parceira. Como se trata de um modelo dinâmico, a verificação de emparelhamento entre 2 fichas deve se dar também considerando o rótulo das duas fichas, ou seja, duas fichas são consideradas "emparelhadas" se e somente se elas se dirigirem para a mesma instrução *e* se tiverem rótulos iguais.

Quando a(s) ficha(s) chega à UP, é verificado qual é a instrução que agirá sobre elas (as fichas contém somente o "endereço" da instrução) e para onde (para que instrução) os resultados da instrução deverão ir. Instruções da MDFM podem produzir nenhum, um ou dois resultados diferentes; no caso de apenas um resultado ser produzido, ele poderá ir para uma ou duas instruções.

O pacote com as fichas mais a instrução mais os endereços de destino dos resultados é enviado para as unidades funcionais (UFs); as UFs são equivalentes e a instrução é executada na primeira UF disponível. As fichas produzidas são enviadas, através da chave, para a UA de onde irão para a UE e o ciclo se repete. Quando o (s) resultado (s) é produzido ele é enviado para fora pela chave.

O anel forma um elemento processador EP; muitos EP podem ser construídos todos compartilhando a mesma chave como mostrado na figura 8.2.

8.2.1 Formato das Fichas

A ficha na MDFM é composta por 96 bits. Destes, 32 bits são de dados, 36, de rótulo contendo 3 campos de 12 bits, (índice, iteração, nome de ativação) 16 para endereçamento, 4 para funções de emparelhamento, 3 para o tipo da ficha, e alguns bits para expansões.

Note que somente 30% da ficha carrega informação útil; o restante é "overhead" necessário para o correto funcionamento do sistema.

8.2.2 Formato das Instruções

Uma, instrução típica da MDFM, pode ser vista a seguir. Note que essa representação é mantida na UP.

1 5NNOT1 7LEW1 8LBY

Essa instrução, cujo endereço é 1 5 (1 é o segmento e 5 o offset: 16 segmentos são possíveis com 4096 posições cada) é uma negação lógica (NOT). Ela produz só um resultado lógico (N de normal) e envia duas cópias (2 fichas com campo de dados idênticos) para duas instruções diferentes: a 1 7 e a 1 8. A instrução 1 7 é binária e a ficha produzida pela instrução NNOT será a sua entrada esquerda; a instrução 1 8 é unária e recebe apenas uma ficha de entrada.

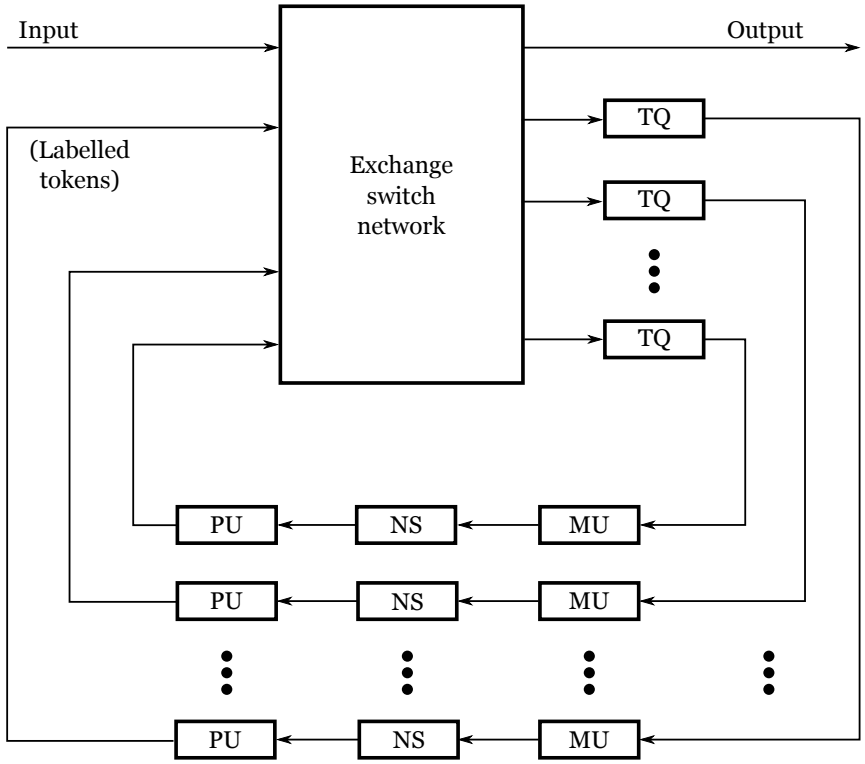


Figura 8.2: Múltiplos EP

8.3 Exercício

Considere o programa em linguagem de máquina da MDFM dado anexo. Construa o grafo de fluxo de dados correspondente e descubra o que faz o programa.

Capítulo 9

Software para Fluxo de Dados

9.1 Introdução

O fluxo de dados é um modelo computacional completamente novo se comparado ao modelo computacional von Neumann. É natural, pois, que para máquinas de fluxos de dados, programas sejam escritos em linguagens não convencionais, ou seja, diferentes de FORTRAN, PASCAL, C etc. Apesar de linguagens funcionais ("ou aplicativas") poderem ser executadas em máquinas de fluxo de dados, linguagens específicas para fluxo de dados surgiram tais como VAL, Id e Sisal. Antes porém, de se estudar essas linguagens, vejamos porque as convencionais são insatisfatórias.

9.2 Crítica de Linguagens von Neumann

Tavez a melhor crítica que se possa fazer às linguagens convencionais é aquela que John Backus, o inventor do FORTRAN fez em 1978.

> I think conventional languages are for the birds. They're really low-level languages. They're just extensions of the von Neumann computer, and they keep our noses in the dirt of dealing with individual words and computing addresses, and doing all kinds of silly things like that, things that we've picked up from programming for computers; we've built them into programming languages; we've built them into FORTRAN; we've built them into PL/1; we've built them into almost every language. The only languages that broke free from that are LISP and APL, and in my opinion they haven't gone far enough.

Backus explica que variáveis em linguagens convencionais têm uma relação direta com as células de armazenamento de dados num computador convencional e que os comandos de atribuição nada mais são do que uma imitação do mecanismo de acesso a essas memórias. Ele acredita que já é hora de se liberar desse estilo e propõe uma linguagem funcional chamada FP.

Aparentemente, o mal é que as linguagens são de nível muito baixo ou seja muito próximos da máquina mas muito distantes do programador.

Assim, o programador fica mais preocupado no "como" fazer do que no "o que" fazer.

Para se entender a diferença entre as várias linguagens é interessante que se defina o que é "variável" nas diferentes linguagens.

Na matemática uma variável é uma quantidade cujo valor não é sabido no momento. Entretanto, seu valor é único e uma vez "descoberto" não pode ser "alterado". Este é aproximadamente o significado em linguagens funcionais. Porém em linguagens imperativas, variáveis podem não só ser lidas por outras partes do programa como também podem ser "modificados".

Este conceito é chamado de "efeito colateral" (side-effect) que pode tornar a exploração de paralelismo de um programa consideràvelmente mais difícil. Note por outro lado que o mecanismo é bastante eficiente em termos de "passagem de mensagens" de uma parte de um programa para outra.

9.3 Linguagens de atribuição única

Linguagens funcionais podem ser compiladas para grafos de fluxo de dados. Estes grafos são então executados de forma faminta ("eager evaluation"). Lembrem-se que o enfoque da redução favorece uma execução "vagabunda" ("lazy") do mesmo grafo.

Apesar da linguagem VAL, que é utilizada no projeto de fluxo de dados estático do MIT ser funcional, esse não é o caminho normal das linguagens para máquina de fluxo de dados. De fato, a maioria das outras linguagens (Id, LAU, SISAL) não são estritamente funcionais; elas são chamadas de linguagens de atribuição única (single assignment language) pois permitem que o valor de uma variável seja atribuído uma vez. Isso é feito para otimizar o tratamento de estruturas de dados que normalmente devem ser armazenadas em memórias convencionais do tipo von Neumann.

Normalmente também, essas linguagens não permitem a passagem de funções como parâmetros o que é comum em linguagens funcionais puras. Não obstante, linguagens de atribuição única não permitem efeitos colaterais e consequentemente, são facilmente compilados para um grafo de fluxo de dados.

9.3.1 SISAL

A linguagem SISAL (Stream and Iteration in a Single Assignment Language) foi resultado de um esforço conjunto da DEC (Digital Equipment Corporation), da Universidade de Manchester e da Universidade de Colorado e tinha como um dos objetivos, ser uma linguagem comum a ser utilizado em várias arquiteturas paralelas a fim de compará-las.

Construções Básicas

Uma expressão em SISAL pode retornar N resultados. Assim a expressão 1, 2, 3 retorna 3, constantes inteiras. O número de resultados retornados é chamado de "aridade" ("arity").

A computação condicional é expressa pelo comando "if":

```
if b(x) them f(x) else g(x) end if
```

Note que b(x) deve retornar um valor lógico e que f(x) e g(x) devem ter mesma aridade e tipos de resultados. Para definir valores de variáveis é possível usar-se a construção "let":

```
let
       a: = 1;
       b: = 2
in
       a, a + b
end let
```

que tem aridade dois.

Dois tipos de ciclos repetitivos são definidos: o paralelo e o iterativo. O paralelo ou "forall" é

```
for i in 1, 10
   el: = a[i] + b[i]
returns array of el
end for
```

O ciclo interativo seria (um exemplo extremamente artificial):

```
for initial
    i = 1
repeat
    i: = old(i) + 1
until i = 20
returns value of i
end for
```

Note que, neste caso, o valor de i é "re-atribuído" mas não existe perda de funcionalidade devido à introdução do operador "old" que faz referência à instância anterior do valor de i (do ciclo anterior).

Finalmente, a mais importante das contruções, a função:

```
function double( X: integer returns integer)
     2*X
end function
```

Capítulo 10

Problemas do Modelo de Fluxo de Dados

10.1 Introdução

O modelo de fluxo de dados apresenta-se bastante promissor na medida em que pode explorar, sem interferência do programador, praticamente todo o paralelismo existente numa certa aplicação (pelo menos, todo o paralelismo a nível de instrução e acima). Máquinas baseadas no modelo de fluxo de dados apresentam "speedups" melhores do que as máquinas paralelas convencionais; mesmo com 128 anéis (cada um com 20 unidades funcionais o que dá $128x20 = 2560$ unidades trabalhando simultaneamente) a máquina de Manchester apresenta eficiência maior que 60

Máquinas de fluxo de dados, por outro lado, apresentam muitos problemas que não existem em máquinas convencionais. A seguir serão brevemente descritos os principais problemas encontrados nestas máquinas.

10.2 Código Sequencial

Todo programa de computador tem partes intrinsicamente sequenciais, ou seja, partes que não podem ser paralelizadas de forma nenhuma.

Estes blocos sequenciais de código precisam ser executados rápidamente para que o tempo total de execução de um programa não acabe ficando muito longo.

Máquina de fluxo de dados, infelizmente, tendem a ser extremamente ineficientes na execução de código sequencial. Este não parece ser um problema insolúvel mas até o momento, nenhuma máquina de fluxo de dados proposta conseguiu corrigir o problema.

10.3 Estrutura de Dados

As arquiteturas de von Neumann são bastante esficientes no tratamento de estrutura de dados: de fato, com a introdução do conceito de "pipeline" e de "processamento vetorial", o processamento de estrutura de dados em máquinas convencionais pode ser considerado bastante eficiente.

No modelo de fluxo de dados porém, a idéia de armazenamento de dados não existe e o tratamento de estruturas de dados pode ser extremamente ineficiente. Possíveis soluções

estão sendo analisadas e alguns pesquisadores acreditam que máquinas de fluxo de dados que incorporem de alguma forma o conceito de armazenamento de dados (como a MDFM que possui uma "structure store" ou a máquina dinâmica do MIT com suas I-structures) podem ser tão eficientes quanto máquinas convencionais; isso porém ainda precisa ser provado.

10.4 Uso excessivo de memória

Este é um problema típico do fluxo de dados dinâmico que não existe em outras máquinas. Na verdade ele apareceu (até certo ponto, surpreendentemente) nas primeiras simulações da arquitetura de MDFM e da máquina dinâmica do MIT.

O problema é o seguinte: na execução de muitos programas, tipicamente os que apresentam fases com alto grau de paralelismo, o uso da unidade de armazenamento e da unidade de emparelhamento (no caso da MDFM) pode ser muito grande chegando facilmente à condição de "overflow". Em função disso, o tamanho dessas unidades deve ser projetado para alta capacidade de armazenamento o que encarece a máquina (principalmente no caso da unidade de emparelhamento que tende a ser a unidade mais cara).

A razão desse gasto excessivo de memória vem do "excesso de paralelismo" que é exposto em ambientes de fluxo de dados. Na verdade, se o paralelismo de uma aplicação for muito mais alto que aquele que pode ser explorado pela máquina, a consequência será um armazenamento excessivo de fichas nas unidades de armazenamento e emparelhamento.

Para solucionar o problema é necessário algum método de controle de paralelismo. Tais métodos têm sido sugeridos recentemente por pesquisadores na área.

10.5 Depuração

O problema de se depurar programas em máquinas paralelas é reconhecidamente difícil. Máquinas de fluxo de dados não são exceção. O problema ocorre porque o "controle" em máquinas paralelas, incluindo as de fluxo de dados é muito menor do que o encontrado em máquinas de von Neumann; assim, a interrupção de um programa em fluxo de dados não é fácil de ser implementada. Também, como a ordem de execução das instruções não é determinística, um depurador pode dar resultados diferentes em duas execuções diferentes.